PROYECTOS ÁGILES CON SCRUM

Flexibilidad, aprendizaje, innovación y colaboración en contextos complejos.



Alaimo, Diego Martín
Proyectos ágiles con Scrum : flexibilidad, aprendizaje, innovación y colaboración en contextos complejos . - 1a ed. - Ciudad Autónoma de Buenos Aires : Kleer, 2013. E-Book.

ISBN 978-987-45158-1-0

1. Informática. 2. Gestión. I. Título
CDD 005.3

Diseño de tapa: Martín Alaimo
Revisión y feedback: Daniela Casquero, Juan Gabardini, Constanza Molinari, Carlos Peix, Florencia Poutón, Pablo Tortorella, Martín Salías y Claudia Sandoval.

1ª edición: octubre de 2013

Ediciones Kleer – http://www.kleer.la
publicamos@kleer.la
Tucumán 373, 1er Piso
Cdad. Autónoma de Buenos Aires, 1049, Argentina

© 2013 Martín Alaimo
Contacto autor: martin.alaimo@kleer.la

Queda hecho el depósito que establece la ley 11.723

A la comunidad ágil latinoamericana, por sus aportes, apoyo, revisiones y espíritu de camaradería. A mis colegas en Kleer, por permitirme aprender día a día. A Daniela, por el empuje y acompañamiento en este viaje.

Contenido

1. De la secuencialidad a la iteración 7
El fracaso del modelo en cascada 7
El origen de las Metodologías Ágiles 12
Manifiesto Ágil 13

2. Scrum 17
Cynefin: la complejidad que nos rodea 17
Dominio Simple 18
Dominio Complicado 18
Dominio Complejo 19
Dominio Caótico 19
Dominio Desordenado 20
Y entonces… ¿qué es Scrum? 20
Principios de Scrum 22
Valores de Scrum 24
Roles de Scrum 25
Product Owner 25
Equipo de Desarrollo 27
ScrumMaster 29
Elementos de Scrum 33
Product Backlog 33
Sprint Backlog 40
Incremento funcional potencialmente entregable 41
Dinámica (flujo del trabajo) 41
Sprint (Iteración) 42
Sprint Planning Meeting (Planificación de Sprint) 43
Scrum Diario 47
Revisión de Sprint 50
Retrospectiva 51
Refinamiento del Product Backlog 53

3. Desarrollo Evolutivo 54
Creación Evolutiva 54

Minimum Viable Product 56
Minimum Marketable Features 57
Visual Story Mapping 57
Proceso de Análisis Ágil 59
Roles de Usuario 59
Visual Story Mapping en la Práctica 71
Identificación de los Procesos de Negocio 71
Identificación de Funcionalidades del Software (Herramientas) 71
Identificación de MVP y posteriores entregas 76

4. Historias de Usuario 82
Componentes de una Historia de Usuario 83
Redacción de una Historia de Usuario 84
INVEST - Características de una Historia de Usuario 86
Independientes (I) 86
Negociable (N) 86
Valorable (V) 87
Estimable (E) 88
Pequeña (Small) 88
Verificable (Testable) 89
Definición de Listo 90
Definición de Terminado 90
Las Historias de Usuario de Nuestro Producto 92

5. Estimaciones Ágiles 101
Cono de la Incertidumbre 101
Estimaciones en contextos inciertos 103
Escalas de PBIs y Estimaciones 104
Métodos Delphi de Predicción y Estimación 107
Planning Poker 108
La Sabiduría de las Multitudes (Wisdom of Crowds) 110
Conclusiones sobre estimaciones Ágiles 111

6. Plan de Entregas (*Release Plan*) 112
Incepción 118
Duración del Proyecto 118

7. Costo del Proyecto..... 120
Bibliografía Recomendada..... 121
Acerca del Autor..... 122
Acerca de Kleer..... 123

1. De la secuencialidad a la iteración

El fracaso del modelo en cascada

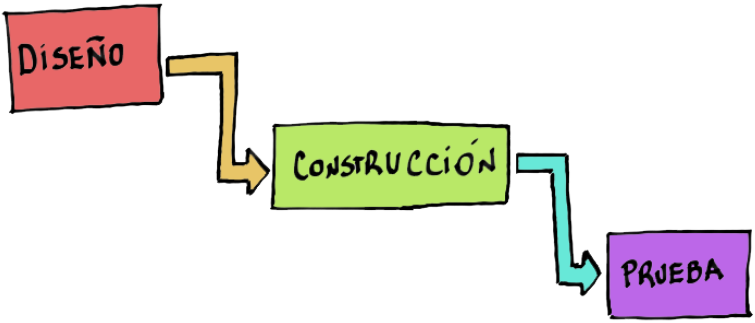


Ilustración 1: Modelo en cascada

El desarrollo de software no es una disciplina sencilla. En las últimas décadas los lenguajes estructurados modernos[1], de modelado (UML)[2] y posteriormente varias herramientas[3] intentaron sin éxito posicionarse como las "balas de plata"[4] para resolver algunos de sus problemas. Incluso se había llegado a contar con herramientas poderosas y procesos de modelado y

[1] Edward Yourdon, *Análisis Estructurado Moderno*, Prentice-Hall, 1993
[2] Unified Modeling Language (UML), Information technology, ISO/IEC 19501:2005 http://www.uml.org
[3] Herramientas CASE (U-CASE, M-CASE, L-CASE): http://es.wikipedia.org/wiki/Herramienta_CASE
[4] El término ha sido adoptado como metáfora referida a cualquier solución sencilla que tiene una eficacia extrema. Suele aparecer con la expectativa de que algún nuevo desarrollo tecnológico o la práctica fácil de implementar resuelva alguno de los problemas vigentes principales. Fuente: Wikipedia

diseño sin tener muy en claro cómo aplicarlos a la hora de construir el software.

No fue sino hasta la adopción amplia y consciente de un tercer elemento injustamente relegado a un papel secundario, las metodologías de desarrollo, que se encontraron soluciones adecuadas a muchos problemas. La mayoría de estas metodologías fueron introducidas desde la ingeniería civil, lo que resultó en un exhaustivo control sobre los procesos y las tareas.

El Modelo Secuencial de Procesos, también conocido como Waterfall Model o Modelo en Cascada, se convirtió en el modelo metodológico más utilizado dentro de la industria. Data de principios de los años setenta y tiene sus orígenes en los ámbitos de la manufactura y la construcción, ambientes físicos altamente rígidos donde los cambios se vuelven prohibitivos desde el punto de vista de los costos, sino prácticamente imposibles. Como no existía proceso alguno en la industria del software, estap condición no impidió su adopción.

La primera mención pública (reconocida) de este tipo de metodologías fue realizada en un artículo que data de 1970 donde el Dr. Winston W. Royce[5] presenta -sin mencionar la palabra "Waterfall"- un modelo secuencial para el desarrollo de software que comprendía las siguientes fases:

- Especificación de requerimientos
- Diseño

[5] Winston Royce, *Managing the Development of Large Software Systems*, Proceedings of IEEE WESCON 26, 1970: 1–9

- Construcción (también conocida como implementación o codificación)
- Integración
- Verificación o prueba y debugging
- Instalación
- Mantenimiento

El proceso Waterfall sugiere una evolución secuencial. Por ejemplo: primero se realiza la fase de especificación de requerimientos. Una vez que se encuentra completa se procede a un "sign-off" (firma/aprobación) que *congela* dichos requerimientos, y es recién aquí cuando se inicia la fase de diseño del software, fase donde se produce un plano o "blueprint" del mismo para que los codificadores/programadores lo implementen.

Hacia el final de la fase de implementación, diferentes componentes desarrollados son integrados con el fin de pulir las interfaces entre ellos. El siguiente paso es la fase de verificación en la que los testers someten el sistema a diferentes tipos de pruebas funcionales mientras los programadores corrigen el código donde sea necesario. Una vez que el sistema responde satisfactoriamente a la totalidad de las pruebas, se inicia una etapa de instalación y mantenimiento posterior.

Los problemas detectados en los modelos tradicionales o de tipo Waterfall se fundamentan, por un lado, en el entorno altamente cambiante propio de la industria, y por el otro, en el proceso mismo de desarrollo de software donde el resultado depende de la actividad cognitiva de las personas más que de las prácticas y controles empleados.

A medida que han pasado los años, y con el advenimiento de las economías globalizadas y los entornos web, el contexto de negocio de los sistemas ha pasado de ser relativamente estable a convertirse en un contexto altamente volátil, donde los requerimientos expresados hoy, en muy pocas oportunidades son válidos unos meses más tarde. Bajo esta nueva realidad, las metodologías Waterfall resultaron muy "pesadas" y prohibitivas para responder satisfactoriamente a los cambios de negocio.

En el año 1994 el Standish Group publicó un estudio conocido como el "CHAOS Report"[6] donde se encontró la siguiente tasa de éxito en los proyectos de desarrollo de software en general:

- 31.1% es cancelado en algún punto durante el desarrollo del mismo
- 52.7% es entregado con sobrecostos, en forma tardía o con menos funcionalidades de las inicialmente acordadas
- 16.2% es entregado en tiempo, dentro de los costos y con las funcionalidades comprometidas

Los datos publicados, entre otros, mostraron estos índices:

Sobrecostos	% de Respuestas
Menos del 20%	15.5%
21 - 50%	31.5%
51 - 100%	29.6%
101 - 200%	10.2%
201 - 400%	8.8%
Mayor al 400%	4.4%

[6] The CHAOS Report (1994), Standish Group - http://www.standishgroup.com/sample_research/chaos_1994_1.php

Funcionalidad entregada	% de Respuestas
Menos del 25%	4.6%
25 - 49%	27.2%
50 - 74%	21.8%
75 - 99%	39.1%
100%	7.3%

Factores mas importantes para el éxito de un proyecto	% de Respuestas
Involucramiento del usuario	15.9%
Apoyo de la gerencia	13.9%
Claridad en los requerimientos	13.0%
Planificación apropiada	9.6%
Expectativas realistas	8.2%
Hitos más acotados	7.7%
Personal competente	7.2%
Compromiso	5.3%
Objetivos y visión claros	2.9%
Staff enfocado y dedicado	2.4%
Otros	13.9%

Factores mas comunes de cancelación de proyectos	% de Respuestas
Requerimientos incompletos	13.1%
Falta de involucramiento del usuario	12.4%
Falta de recursos	10.6%
Expectativas irreales	9.9%
Falta de soporte gerencial	9.3%
Requerimientos y especificaciones cambiantes	8.7%
Falta de planificación	8.1%
No se necesitaba más	7.5%
Falta de gestión IT	6.2%
Analfabetismo técnico	4.3%
Otros	9.9%

Factores mas importantes de desafío para los proyectos	% de Respuestas
Falta de input del usuario	12.8%
Requerimientos y especificaciones incompletas	12.3%
Requerimientos y especificaciones cambiantes	11.8%
Falta de apoyo gerencial	7.5%
Falta de conocimientos técnicos	7.0%
Falta de recursos	6.4%
Expectativas irreales	5.9%
Objetivos poco claros	5.3%
Calendario poco realista	4.3%
Nuevas tecnologías	3.7%
Otros	23.0%

Demora	% de Respuestas
Menos del 20%	13.9%
21 - 50%	18.3%
51 - 100%	20.0%
101 - 200%	35.5%
201 - 400%	11.2%
Mayor al 400%	1.1%

Las conclusiones de la investigación sugieren que el involucramiento del usuario y el empleo de periodos de tiempo más cortos son claves para incrementar las tasas de proyectos exitosos

Bajo este contexto surgieron nuevas metodologías, como por ejemplo:

- Metodologías en Espiral
- Metodologías Iterativas
- Metodologías Ágiles

Tanto las Metodologías en Espiral como las Metodologías Iterativas se encuentran fuera del alcance de este trabajo, por lo que pasaremos directamente a entender las Metodología Ágiles.

El origen de las Metodologías Ágiles

En los ‘90s surgieron varios movimientos identificados con el nombre de Metodologías Livianas (*Lightweight Methodólogies*). Entre estos se encuentran Extreme Programming (XP), Scrum, Software Craftmanship, Lean Software Development, etc.

Más tarde, en febrero de 2001, se reunieron en Utah (EEUU) un grupo de diecisiete profesionales reconocidos del desarrollo de software, y referentes de las metodologías livianas existentes al momento, con el objetivo de determinar los valores y principios

que les permitirían a los equipos desarrollar software de forma más acertada con las necesidades del cliente y responder mejor a los cambios que pudieran surgir a lo largo de un proyecto de desarrollo. Se pretendía ofrecer una alternativa a los procesos de desarrollo de software tradicionales, caracterizados por la rigidez y dominados por la documentación.

En esta reunión se creó la Agile Alliance[7], una organización sin fines de lucro cuyo objetivo es el de promover los valores y principios de la filosofía ágil y ayudar a las organizaciones en su adopción. También se declaró la piedra angular del movimiento ágil, conocida como Manifiesto Ágil (*Agile Manifesto*[8]).

Manifiesto Ágil



Ilustración 2: Manifiesto Ágil, Utah, 2001

El Manifiesto Ágil se compone de 4 valores y 12 principios.

[7] http://www.agilealliance.org
[8] http://www.agilemanifesto.org

Valores

1. Valorar a las personas y las interacciones entre ellas por sobre los procesos y las herramientas

Las personas son el principal factor de éxito de un proyecto de software. Es más importante construir un buen equipo que construir el contexto. Muchas veces se comete el error de construir primero el entorno de trabajo y esperar que el equipo se adapte automáticamente. Por el contrario, la agilidad propone crear el equipo y que éste construya su propio entorno y procesos en base a sus necesidades.

2. Valorar el software funcionando por sobre la documentación detallada

La regla a seguir es "no producir documentos a menos que sean necesarios de forma inmediata para tomar un decisión importante". Estos documentos deben ser cortos y centrarse en lo esencial. La documentación (diseño, especificación técnica de un sistema) no es más que un resultado intermedio y su finalidad no es dar valor en forma directa al usuario o cliente del proyecto. Medir avance en función de resultados intermedios se convierte en una simple "ilusión de progreso".

3. Valorar la colaboración con el cliente por sobre la negociación de contratos

Se propone que exista una interacción constante entre el cliente y el equipo de desarrollo. Esta mutua colaboración será la que dicte la marcha del proyecto y asegure su éxito.

4. Valorar la respuesta a los cambios por sobre el seguimiento estricto de los planes

La habilidad de responder a los cambios que puedan surgir a lo largo del proyecto (cambios en los requisitos, en la tecnología, en el equipo, etc.) determina también su éxito o fracaso. Por lo tanto, la planificación no debe ser estricta sino flexible y abierta.

Principios

Los valores anteriores son los pilares sobre los cuales se construyen los doce principios del Manifiesto Ágil. De estos doce principios, los dos primeros son generales y resumen gran parte del espíritu ágil del desarrollo de software, mientras que los siguientes son más específicos y orientados al proceso o al equipo de desarrollo:

1. Nuestra mayor prioridad es satisfacer al cliente a través de entregas tempranas y frecuentes de software con valor.
2. Aceptar el cambio incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan los cambios para darle al cliente ventajas competitivas.
3. Entregar software funcionando en forma *frecuente*, desde un par de semanas a un par de meses, prefiriendo el periodo de tiempo más corto.
4. Expertos del negocio y desarrolladores deben trabajar juntos diariamente durante la ejecución del proyecto.
5. Construir proyectos en torno a personas motivadas, generándoles el ambiente necesario, atendiendo sus necesidades y confiando en que ellos van a poder hacer el trabajo.
6. La manera más eficiente y efectiva de compartir la información dentro de un equipo de desarrollo es la conversación cara a cara.

7. El *software funcionando* es la principal métrica de progreso.
8. Los procesos ágiles promueven el desarrollo sostenible. Los sponsors, desarrolladores y usuarios deben poder mantener un ritmo constante indefinidamente.
9. La atención continua a la excelencia técnica y buenos diseños incrementan la agilidad.
10. La simplicidad –el arte de maximizar la cantidad de trabajo no hecho- es esencial.
11. Las mejores arquitecturas, requerimientos y diseños emergen de equipos auto-organizados.
12. A intervalos regulares, el equipo reflexiona acerca de cómo convertirse en más efectivos, luego mejora y ajusta su comportamiento adecuadamente.

2. Scrum

Cynefin: la complejidad que nos rodea

Antes de hacer hincapié en Scrum en sí mismo, veamos el contexto para el cual Scrum es más eficiente. Podemos utilizar el marco Cynefin[9] para comprender las diferentes situaciones en las que nos podemos encontrar operando, y cuál es, según este enfoque, la manera más eficiente de responder a cada una de ellas.

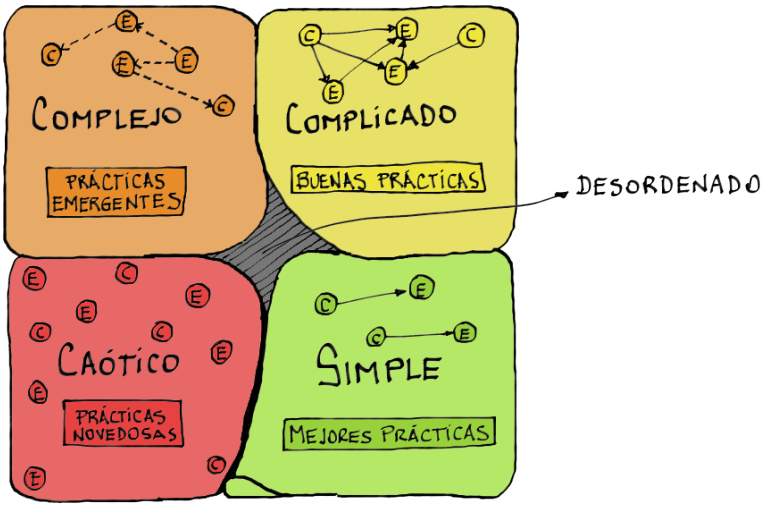


Ilustración 3: Marco Cynefin

El marco Cynefin compara las características de cinco dominios de complejidad diferentes: simple, complicado,

[9] Snowden & Boone, *A Leader's Framework for Decision Making*, BHR, 2007

complejo, caótico y desordenado, en el centro (ver Ilustración 3). Analicemos cada uno de ellos.

Dominio Simple

En este dominio se opera con problemáticas simples. Es muy fácil identificar las causas y sus efectos. Por lo general, la respuesta correcta es clara, conocida por todos e indiscutible. En este dominio existen las mejores prácticas, soluciones conocidas para problemas conocidos. Los procesos más eficientes en este dominio son aquellos que especifican una serie lógica de pasos y se ejecutan de manera repetitiva, una y otra vez. Ejemplos de este dominio son la construcción en serie de un mismo producto, la instalación en muchos clientes de un mismo sistema. Si bien Scrum puede funcionar en este contexto, los procesos compuestos por pasos bien definidos son mucho más eficientes.

Dominio Complicado

En este dominio encontramos problemas complejos, buenas prácticas y perfiles expertos. Hay múltiples soluciones correctas para una misma problemática, pero se requiere del involucramiento de expertos para poder identificarlas. Un ejemplo típico de este escenario es la solución de un problema de performance en un software o base de datos, la sincronización de semáforos en un cruce de 3 avenidas, la búsqueda de eficiencia en la distribución logística de mercaderías, etc. Si bien Scrum podría emplearse, no necesariamente sea la forma más eficiente de resolver estas situaciones, donde funcionaría mejor un conjunto de expertos en la materia que releven la situación, investiguen diferentes alternativas y planteen la solución en base a las

buenas prácticas. Una práctica habitual de este dominio es el mantenimiento de sistemas y soporte técnico.

Dominio Complejo

Cuando nos enfrentamos a problemas complejos, los resultados se vuelven más impredecibles. No existen ni mejores ni buenas prácticas catalogadas para las situaciones frente a las cuales nos podemos encontrar. Simplemente, no sabemos con anticipación si una determinada solución va a funcionar. Solo podemos examinar los resultados y adaptarnos. Este es el dominio de las prácticas emergentes. Las soluciones encontradas rara vez son replicables, con los mismos resultados, a otros problemas similares. Para poder operar en la complejidad necesitamos generar contextos donde haya lugar para la experimentación y donde el fallo sea de bajo impacto. Se requieren niveles altos de creatividad, innovación, interacción y comunicación. El desarrollo de nuevos productos o la incorporación de nuevas características en productos existentes es un contexto complejo en el que Scrum se utiliza mucho para actuar, inspeccionar y adaptar las prácticas emergentes de un equipo de trabajo.

Dominio Caótico

Los problemas caóticos requieren una respuesta inmediata. Estamos en crisis y necesitamos actuar de inmediato para restablecer cierto orden. Imaginemos que el sistema de despacho de vuelos en un aeropuerto de alto tráfico deja de funcionar. Este no sería un escenario para utilizar Scrum, aquí debemos actuar de inmediato, alguien debe tomar el control y mover la situación fuera del caos. Por ejemplo,

solucionar el problema inmediatamente (sin importar la forma técnica), para luego, fuera del caos, evaluar y aplicar una solución más robusta, de ser necesario. Este es el dominio de la improvisación.

Dominio Desordenado

Nos movemos en el espacio desordenado cuando no sabemos en qué dominio estamos. Se la clasifica como una zona peligrosa, ya que no podemos medir las situaciones ni determinar la forma de actuar. Es muy típico en estas situaciones que las personas interpreten las situaciones y actúen en base a preferencias personales. El gran peligro del dominio desordenado es actuar de manera diferente a la que se necesita para resolver ciertos problemas. Por ejemplo, mucha gente en el ámbito del desarrollo de software está acostumbrada al desarrollo secuencial, por fases, detalladamente planificado utilizando las mejores prácticas de la industria, y este enfoque, que corresponde al dominio Simple, muchas veces se aplica en el dominio complejo. Si nos encontráramos en el espacio desordenado, todo lo que hagamos debe estar enfocado netamente a salirnos de ese espacio hacia uno mejor identificado, para luego actuar de la manera en que dicho dominio lo requiera.

Y entonces… ¿qué es Scrum?

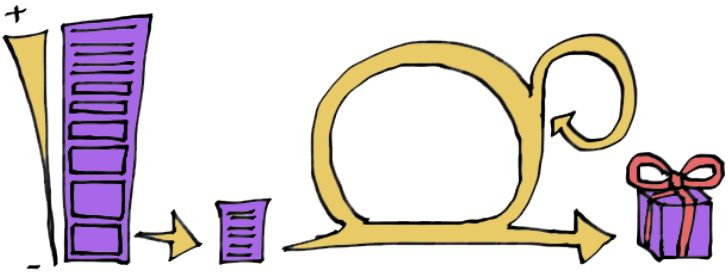


Ilustración 4: Scrum

Scrum es un marco de trabajo que nos permite encontrar prácticas emergentes en dominios complejos, como la gestión de proyectos de innovación. No es un proceso completo, y mucho menos, una metodología. En lugar de proporcionar una descripción completa y detallada de cómo deben realizarse las tareas de un proyecto, genera un contexto relacional e iterativo, de inspección y adaptación constante para que los involucrados vayan creando su propio proceso. Esto ocurre debido a que no existen ni mejores ni buenas prácticas en un contexto complejo. Es el equipo de involucrados quien encontrará la mejor manera de resolver sus problemáticas. Este tipo de soluciones serán emergentes.

El equipo de desarrollo se encuentra apoyado en dos roles: el ScrumMaster y el Product Owner. El ScrumMaster es quien vela por la utilización de Scrum, la remoción de impedimentos y asiste al equipo a que logre su mayor nivel de performance posible. Puede ser considerado un coach o facilitador encargado de acompañar al equipo de desarrollo. El Product Owner es quien representa al negocio, *stakeholders*, cliente y usuarios finales. Tiene la responsabilidad de conducir al equipo de desarrollo hacia el producto adecuado.

El progreso de los proyectos que utilizan Scrum se realiza y verifica en una serie de iteraciones llamadas Sprints. Estos Sprints tienen una duración fija, pre-establecida de no más de un mes. Al comienzo de cada Sprint el equipo de desarrollo realiza un compromiso de entrega de una serie de funcionalidades o características del producto en cuestión.

Al finalizar el Sprint se espera que estas características comprometidas estén terminadas, lo que implica su análisis, diseño, desarrollo, prueba e integración al producto. En este momento es cuando se realiza una reunión de revisión del producto construido durante el Sprint, donde el equipo de desarrollo muestra lo construido al Product Owner y a cualquier stakeholder interesado en participar. El feedback obtenido en esta reunión puede ser incluido entre las funcionalidades a construir en futuros Sprints.

Principios de Scrum

Scrum es el modelo más utilizado dentro de las Metodología Ágiles. Muchos de los valores y principios del Manifiesto Ágil tienen su origen en Scrum. Revisemos, ahora desde la perspectiva de Scrum, los valores del Manifiesto Ágil:

1. **Individuos e interacciones por sobre procesos y herramientas.** Scrum se apoya en la confianza hacia las personas, sus interacciones y los equipos. Los equipos identifican lo que hay que hacer y toman la responsabilidad de hacerlo, removiendo todos los impedimentos que encuentren en su camino y estén a su alcance. Los equipos trabajan en conjunto con otras partes de la organización cuando los impedimentos están fuera de su ámbito de control.

2. **Software funcionando por sobre documentación exhaustiva.** Scrum requiere que al final de cada Sprint se entregue un producto funcionando. La documentación es entendida, en Scrum, como un producto intermedio sin valor de negocio. Los equipos pueden documentar tanto como crean necesario, pero ninguno de estos documentos pueden ser considerados cono el resultado de un Sprint. El resultado de un Sprint es, nuevamente, el producto funcionando. El progreso del proyecto se mide en base al producto funcionando que se entrega iterativamente.
3. **Colaboración con el cliente por sobre la negociación de contratos.** El Scrum Product Owner es el responsable de la relación que existe con los usuarios finales, *stakeholders* y áreas de la organización que van a obtener el beneficio del producto. El Scrum Product Owner es parte del Equipo Scrum y trabaja colaborativamente con el resto de los individuos dentro del equipo para asegurarse que el producto construido tenga la mayor cantidad posible de valor al final de cada iteración.
4. **Respuesta al cambio por sobre el seguimiento de un plan.** Scrum, por diseño, se asegura que todo el mundo dentro de un equipo tenga toda la información necesaria para poder tomar decisiones informadas sobre el proyecto en cualquier momento. El progreso es medido al final de cada Sprint mediante software funcionando y la lista de características pendientes está visible continuamente

y para todos los miembros. Esto permite que el alcance del proyecto cambie constantemente en función de la retroalimentación provista por los *stakeholders*. Fomentar el cambio es una ventaja competitiva.

Valores de Scrum

Además de los 4 principios mencionados recientemente, Scrum se construye sobre 5 pilares, sus valores:

1. **Foco.** Los Equipos Scrum se enfocan en un conjunto acotado de características por vez. Esto permite que al final de cada Sprint se entregue un producto de alta calidad y, adicionalmente, se reduce el *time-to-market*.
2. **Coraje.** Debido a que los Equipos Scrum trabajan como verdaderos equipos, pueden apoyarse entre compañeros, y así tener el coraje de asumir compromisos desafiantes que les permitan crecer como profesionales y como equipo.
3. **Apertura.** Los Equipos Scrum privilegian la transparencia y la discusión abierta de los problemas. No hay agendas ocultas ni triangulación de conflictos. La sinceridad se agradece y la información está disponible para todos, todo el tiempo.
4. **Compromiso.** Los Equipos Scrum tienen mayor control sobre sus actividades, por eso se espera de su parte el compromiso profesional para el logro del éxito.

5. **Respeto.** Debido a que los miembros de un Equipo Scrum trabajan de forma conjunta, compartiendo éxitos y fracasos, se fomenta el respeto mutuo, y la ayuda entre pares es una cuestión a respetar.

Roles de Scrum

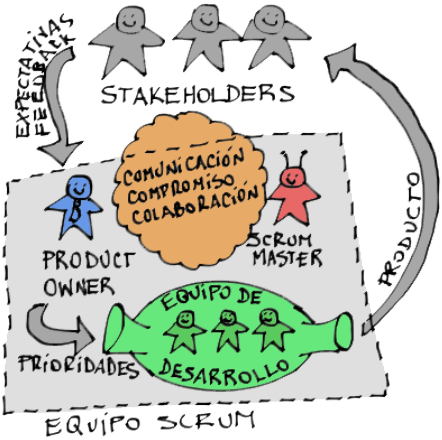


Ilustración 5: Equipo Scrum e interacciones

En un Equipo Scrum se espera que intervengan tres roles: Product Owner, Equipo de Desarrollo y ScrumMaster.

Product Owner

El Product Owner es la persona responsable del éxito del producto desde el punto de vista de los *stakeholders*. Sus principales responsabilidades son:

- Determinar la **visión** del producto, hacia dónde va el equipo de desarrollo
- Gestionar las **expectativas de los stakeholders**

- Recolectar los **requerimientos**
- Determinar y conocer en detalle las **características funcionales** de alto y de bajo nivel
- Generar y mantener el **plan de entregas** (*release plan*): fechas de **entrega** y **contenidos** de cada una
- Maximizar la **rentabilidad** del producto
- Determinar las **prioridades** de cada una de las características por sobre el resto
- Cambiar las prioridades de las características según avanza el proyecto, acompañando así los cambios en el negocio
- Aceptar/rechazar el producto construido durante el Sprint y proveer **feedback** valioso para su evolución
- Participar de la revisión del Sprint junto a los miembros del Equipo de Desarrollo para obtener feedback de los *stakeholders*.



Ilustración 6: Product Owner

El Product Owner se focaliza en maximizar la rentabilidad del producto. La principal herramienta con la que cuenta para poder

realizar esta tarea es la priorización. De esta manera puede reordenar la cola de trabajo del equipo de desarrollo para que éste construya con mayor anticipación las características o funcionalidades más requeridas por el mercado o la competitividad comercial.

Otra responsabilidad importante del Product Owner es la gestión de las expectativas de los stakeholders mediante la comprensión completa de la problemática de negocio y su descomposición hasta llegar al nivel de requerimientos funcionales.

Equipo de Desarrollo

El equipo de desarrollo está formado por todos los individuos necesarios para la construcción del producto en cuestión. Es el único responsable por la construcción y calidad del producto.

El equipo de desarrollo es auto-organizado. Esto significa que no existe un líder externo que asigne las tareas ni que determine la forma en la que serán resueltos los problemas. Es el mismo equipo quien determina la forma en que realizará el trabajo y cómo resolverá cada problemática que se presente. La contención de esta auto-organización está dada por el objetivo a cumplir: transformar las funcionalidades comprometidas en software funcionando y con calidad productiva, o en otras palabras, producir un incremento funcional potencialmente entregable.



Ilustración 7: Equipo de desarrollo

Es recomendable que un equipo de desarrollo se componga de hasta nueve personas[10]. Cada una de ellas debe poseer todas las habilidades necesarias para realizar el trabajo requerido. Esta característica se conoce como multi-funcionalidad y significa que dentro del equipo de desarrollo no existen especialistas exclusivos, sino más bien individuos generalistas con capacidades especiales. Lo que se espera de un miembro de un equipo de desarrollo es que no solo realice las tareas en las cuales se especializa sino también todo lo que esté a su alcance para colaborar con el éxito del equipo.

El equipo de desarrollo tiene tres responsabilidades tan fundamentales como indelegables. La primera es proveer las estimaciones de cuánto esfuerzo será requerido para cada una de las características del producto. La segunda responsabilidad es

10 http://blogs.collab.net/agile/2009/01/17/why-are-scrum-teams-supposed-to-be-small/

comprometerse al comienzo de cada Sprint a construir un conjunto determinado de características en el tiempo que dura el mismo. Y finalmente, también es responsable por la entrega del producto terminado al finalizar cada Sprint.

ScrumMaster

El ScrumMaster es el Coach del equipo y es quien lo ayuda a alcanzar su máximo nivel de productividad posible.

Tomando algunas referencias de Leonardo Wolk podemos decir que el ScrumMaster, en tanto que coach, es un *líder, facilitador, provocador, detective y soplador de brasas*.

Líder por ser un ejemplo a seguir, **facilitador** por fomentar contextos de apertura y discusión donde todos pueden expresar sus opiniones y lograr consensos comunes, **provocador** por desafiar las estructuras rígidas y las antiguas concepciones sobre cómo deben hacerse las cosas, **detective** por involucrarse activamente en la búsqueda e identificación de indicios y pistas en la narrativa del equipo y los individuos y finalmente, **soplador de brasas**, "*un socio facilitador del aprendizaje, que acompaña al otro en una búsqueda de su capacidad de aprender para generar nuevas respuestas*"[11]. Soplar brasas para re-conectar a las personas con sus pasiones, con sus fuegos, muchas veces apagados.

Se espera, además, que el ScrumMaster acompañe al equipo de trabajo en su día a día y garantice que todos, incluyendo al Product Owner, comprendan y utilicen Scrum de forma correcta.

---

[11] Leonardo Wolk, *Coaching – El arte de soplar brasas*, 2003, p. 22-23

Las responsabilidades principales del ScrumMaster son:

- Velar por el **correcto empleo** y **evolución** de Scrum
- Facilitar el **uso de Scrum** a medida que avanza el tiempo. Esto incluye la responsabilidad de que todos asistan a tiempo a las daily meetings, reviews y retrospectivas
- Asegurar que el equipo de desarrollo sea **multi-funcional** y eficiente
- **Proteger** al equipo de desarrollo de distracciones y trabas externas al proyecto
- Detectar, monitorear y **facilitar la remoción de los impedimentos** que puedan surgir con respecto al proyecto y a la metodología[12]
- Asegurar la **cooperación** y **comunicación** dentro del equipo

[12] Estos impedimentos podrán ser resueltos dentro del equipo de desarrollo (usualmente), entre diferentes equipos (*Scrum de Scrums*) o con la intervención de la gerencia.



Ilustración 8: ScrumMaster

Además de estas cuestiones, el ScrumMaster debe detectar **problemas y conflictos interpersonales** dentro del equipo de trabajo. Para respetar la filosofía auto-organizativa del equipo, lo ideal es que el equipo mismo sea quien resuelva estas cuestiones. En el caso de no poder hacerlo, deberá involucrarse al ScrumMaster y eventualmente a niveles más altos de la gerencia.

El ScrumMaster es un Líder Facilitador

No es casualidad la aparición de un nuevo nombre o rol. Este nuevo concepto del enfoque ágil representa el cambio respecto de las responsabilidades y el modelo de gestión de los gerentes de proyectos tradicionales en relación al equipo de trabajo.

El ScrumMaster puede ser visto como un Facilitador o Coach, incluso muchas veces se lo referencia así en lugar de ScrumMaster. Su responsabilidad es asegurar que se cumpla con el proceso de Scrum sin interferir directamente en el desarrollo

del producto final. Es importante establecer que el equipo de Scrum elige la forma de trabajo que más prefiera, siempre que se cumplan las pautas básicas de Scrum, por ello mientras lo hagan no existe una forma "errónea" de trabajar.

El rol del ScrumMaster también incluye asegurar que el desarrollo del producto tenga la mayor probabilidad de ser completado de forma exitosa. Para lograr este cometido, trabaja de cerca con el Product Owner asegurando una correcta priorización de los requerimientos, por un lado, y con el equipo de desarrollo para convertir los requerimientos en un producto funcionando, por el otro.

Por lo que hemos visto, el ScrumMaster tiene un rol más indirecto que un Gerente de Proyectos tradicional, a pesar de esto es un rol vital para el éxito de Scrum. Para todo Gerente de Proyectos tradicional, el cambio hacia esta nueva filosofía de gestión es desafiante. Se dice que "Scrum es fácil, hacer Scrum es difícil[13]". Esta afirmación tiene sus fundamentos en la idea de que una cosa es aprender Scrum y otra muy diferente es aplicar Scrum exitosamente. Emprender este camino significa adoptar una filosofía de liderazgo servil por sobre el comando y control.

Finalmente, cuando un ScrumMaster logra cubrir exitosamente su rol, la implementación de Scrum sucede sin sobresaltos. Las responsabilidades del ScrumMaster deberían cubrir la totalidad de su tiempo. Si bien hay casos en los que el ScrumMaster cumple, además de su rol, el rol de desarrollador, no siempre es la mejor de las situaciones ya que ambas responsabilidades podrían llegar a exceder la disponibilidad de una sola persona, y

13 "Scrum is simple, doing Scrum is hard" - Jim York, CST.

así alguno de ambos roles no estaría siendo cubierto satisfactoriamente.

Elementos de Scrum

El proceso de Scrum posee una mínima cantidad necesaria de elementos formales para poder llevar adelante un proyecto de desarrollo. A continuación describiremos cada uno de ellos.

Product Backlog

El primero de los elementos, y principal de Scrum, es el Backlog del Producto o también conocido como Pila del Producto o Product Backlog.



Ilustración 9: Product Backlog

El Backlog del Producto es básicamente un listado de ítems (Product Backlog Ítems, PBIs) o características del producto a construir, mantenido y priorizado por el Product Owner. Es importante que exista una clara priorización, ya que es esta priorización la que determinará el orden en el que el equipo de

desarrollo transformará las características (ítems) en un producto funcional acabado.

Esta prioridad es responsabilidad exclusiva del Product Owner y, aunque el equipo de desarrollo pueda hacer sugerencias o recomendaciones, es el Product Owner quien tiene la última palabra sobre la prioridad final de los ítems del Product Backlog, teniendo en cuenta el contexto de negocio, el producto mismo y el mercado en el que está inserto.

Priorización por valor de negocio de cada PBI

Una forma de priorizar los ítems del Product Backlog es según su valor de negocio. Podemos entender el valor de negocio como la relevancia que un ítem tiene para el cumplimiento del objetivo de negocio que estamos buscando.

Si planteáramos un ejemplo que ilustre el valor de negocio de los PBIs podríamos decir: en un proyecto cuyo objetivo es aumentar la afluencia de alumnos y facilitar la comunicación de los contenidos de las diferentes carreras de una universidad, se ha decidido crear un sitio web con diferentes características que se encuentran listadas en el Product Backlog. Dos de ellas son 1) que el alumno pueda acceder a los programas de estudios de las diferentes carreras y sus contenidos y 2) que el alumno pueda efectuar el pago en línea de su matrícula y cuotas utilizando una tarjeta de crédito.

En esta situación, muchos podríamos pensar que el requerimiento que implica el pago online con tarjeta de crédito representará un mayor valor de negocio que darle acceso a los alumnos a los contenidos de los programas de estudio, cuando la realidad es a la inversa: 1) el hecho de que un alumno pueda

acceder a los contenidos de los programas de las diferentes carreras aporta un mayor valor hacia el cumplimiento del objetivo del producto (aumentar la afluencia de alumnos e incrementar a comunicación de los programas) que lo que el pago online podría hacer y 2) un alumno podría seguir abonando con tarjeta de crédito telefónicamente.

Priorización por retorno de la inversión (ROI) de cada PBI

Un enfoque diferente de medir la prioridad de un determinado ítem del Backlog es calcular el beneficio económico que se obtendrá en función de la inversión que se deba realizar. Esto, si bien es una simple fórmula matemática, tiene implícita la problemática de encontrar o conocer el valor económico ganado por la incorporación de una determinada característica a un producto. Una vez identificada, el cálculo es relativamente simple:

$$ROI = valor\ de\ negocio/costo$$

Donde el costo representa el esfuerzo necesario para la construcción de una determinada característica de un producto y el valor de negocio es el rédito económico obtenido por su incorporación.

Prioridades según la importancia y el riesgo de cada PBI

Ya sea que los ítems del Backlog se prioricen por valor de negocio o por ROI, en cualquier caso llamémosle "priorizar por importancia", éstos pueden verse complementariamente afectados por el nivel de riesgo asociado a cada uno de ellos.

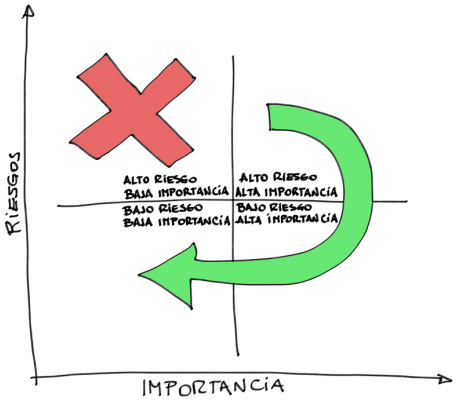


Ilustración 10: Mitigación de riesgos

De esta manera, deberíamos aprovechar la construcción iterativa y evolutiva de Scrum para mitigar riesgos en forma implícita: construyendo primero aquellas características con mayor riesgo asociado y dejando las que poseen menor riesgo para etapas posteriores.

Se recomienda que los PBIs de baja importancia y alto riesgo sean evitados, por ejemplo, transfiriéndolos o eliminándolos del alcance.

Alcance Variable

Debido a que asumimos que no nos es posible conocer de manera anticipada y con un nivel muy fino de detalle todas las características del producto que pretendemos construir, sino que es un viaje de descubrimiento que emprendemos junto con el cliente, este Backlog es un elemento vivo que muta a través del tiempo, al ritmo que vamos aprendiendo sobre el mismo con las entregas iterativas y el *feedback* frecuente.

Si bien, tradicionalmente, el alcance se ha intentado fijar desde el comienzo de un proyecto, y así manejar el costo y el tiempo como los elementos variables, desde la agilidad, esta ecuación se invierte: el tiempo y el costo son los componentes fijos del proyecto, mientras que el alcance es el componente variable.

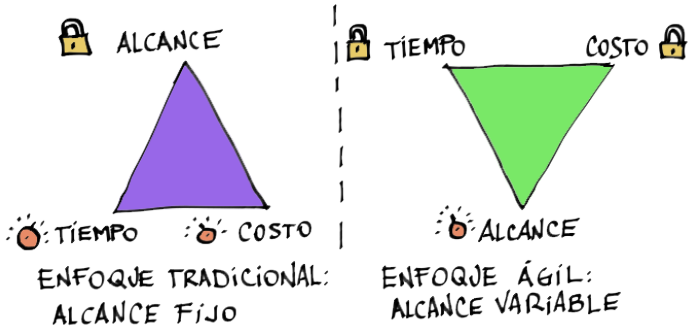


Ilustración 11: Triángulos de hierro

Un Backlog Eficiente

Cuando hablamos de eficiencia, hablamos de obtener el mayor beneficio con el menor esfuerzo posible. Este concepto llevado al Product Backlog significa invertir el esfuerzo de exploración y especificación de la manera más inteligente posible para evitar re-trabajos y desperdicios. Por esto, fomentamos un Product Backlog donde sus ítems más prioritarios están expresados con un nivel de detalle mucho mayor que los ítems de menor prioridad, los cuales están descriptos a un nivel más alto, ya que son los más susceptibles de ser alterados o reemplazados.

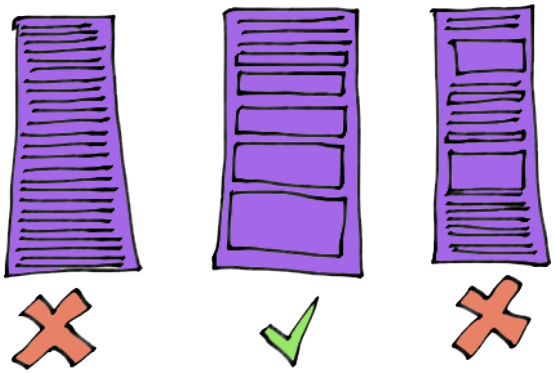


Ilustración 12: Niveles de detalle de PBIs

El Principio de Pareto

Wilfredo Pareto nació en 1848 en Italia, donde creció formando parte de la clase media alta. Fue un reconocido ingeniero, sociólogo, economista, político y filósofo. Uno de sus estudios más reveladores, en aquella época, dejó al descubierto que el 80% de las tierras de Italia pertenecían el 20% de la población. A partir de ese descubrimiento, varios matemáticos y economistas derivaron esas observaciones y las verificaron en otros ámbitos. Uno de ellos fue Joseph Juran, quien en 1941 planteó el Principio de Pareto (o regla del 80/20) aplicado a la calidad: el 80% de los efectos son producidos por el 20% de las causas. Esta ley también se conoció como el principio de "los pocos vitales (el 20% principal que genera el 80% importante) y los muchos triviales (el 80% restante que genera el 20% remanente)".

Aplicando este principio al desarrollo de software, podemos decir que el 20% de las características de un sistema resuelven el 80% de la necesidad de negocio que le da

origen. Y, de manera recursiva, el 20% del 80% restante de las características, resuelven el 80% del 20% restante de negocio. Podemos representar esta relación, recursiva, mediante el siguiente gráfico:

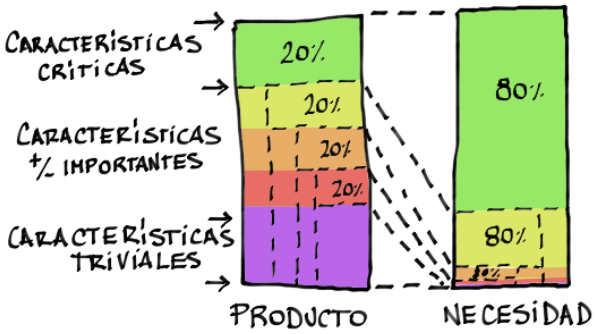


Ilustración 13: Regla del 80/20

Manejo de Contingencias

Aprovechando que el alcance es variable y que todo lo que debemos realizar está priorizado en el Product Backlog según su impacto en el negocio, vamos a utilizar las características menos prioritarias del producto como la contingencia del proyecto frente a imprevistos. Esto quiere decir que, al respetar tiempo y costo, el alcance de menor prioridad sería el que pagaría el precio de retrasos o desvíos. Para que este enfoque sea eficaz, es fundamental la labor del Product Owner y su habilidad para facilitar el descubrimiento de las prioridades por parte de todos los involucrados.

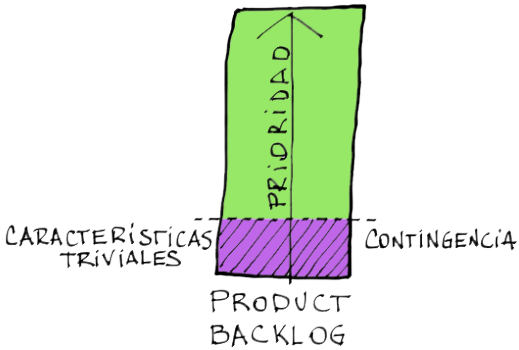


Ilustración 14: Alcance contingente

Sprint Backlog

El Sprint Backlog es el conjunto de PBIs que fueron seleccionados para trabajar en ellos durante un cierto Sprint, conjuntamente con las tareas que el equipo de desarrollo ha identificado que debe realizar para poder crear un incremento funcional potencialmente entregable al finalizar el Sprint.

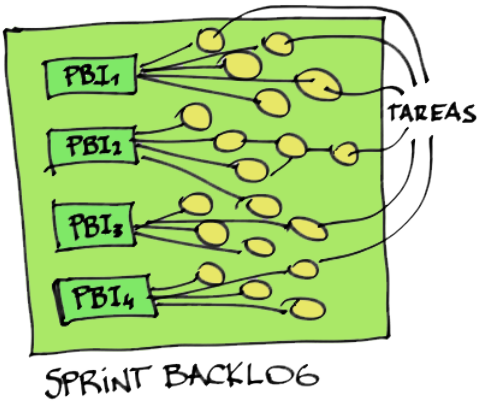


Ilustración 15: Sprint Backlog

Incremento funcional potencialmente entregable

El resultado de cada Sprint debe ser un incremento funcional potencialmente entregable.



Ilustración 16: Incremento Funcional

Incremento funcional porque es una característica funcional nueva (o modificada) de un producto que está siendo construido de manera evolutiva. El producto crece con cada Sprint.

Potencialmente entregable porque cada una de estas características se encuentra lo suficientemente validada y verificada como para poder ser desplegada en producción (o entregada a usuarios finales) si así el negocio lo permite o el cliente lo desea.

Dinámica (flujo del trabajo)

Antes de describir en detalle la dinámica de Scrum, recordemos el mecanismo de timeboxing[14] promovido por Scrum y los principios de ritmo sostenible, entrega frecuente de software valioso y adaptación constante que encontramos en el

14 Ver página 21.

Manifiesto Ágil[15]. La razón es que en conjunto constituyen la piedra angular de la dinámica de Scrum: aprendizaje, inspección y adaptación.

Sprint (Iteración)

Las iteraciones en Scrum se conocen como Sprints. Scrum, como todos los enfoques ágiles, es un proceso de desarrollo incremental e iterativo. Esto significa que el producto se construye en incrementos funcionales entregados en periodos cortos para obtener feedback frecuente.

En general, Scrum recomienda una duración de Sprint de entre 1 y 4 semanas, siendo 2 o 3 semanas lo más habitual que encontraremos en la industria. Una de las decisiones que debemos tomar al comenzar un proyecto o al adoptar Scrum es justamente la duración de los Sprints. Luego, el objetivo será mantener esta duración constante a lo largo del desarrollo del producto, lo que implicará que la duración de una iteración no cambie una vez que sea establecida.

Como excepción podemos mencionar aquellas situaciones donde el equipo mismo decida probar con iteraciones más largas o más cortas. Esta decisión se basa principalmente en la volatilidad del contexto: mientras más volátil sea (negocio cambiante, requerimientos desconocidos, etc.) más corta será la duración del Sprint. Lo importante es recordar que se logra mayor ritmo y previsibilidad teniendo Sprints de duración constante.

15 Ver página 8.

Retrasos y adelantos en un Sprint

Muchas veces podremos encontrar situaciones en donde el equipo de desarrollo se atrase o se adelante. En estos casos, la regla del *timeboxing* no nos permitirá modificar (adelantar o postergar) la fecha de entrega o finalización del Sprint. La variable de ajuste en estos casos será el alcance del Sprint, esto es, en el caso de adelantarnos deberemos incrementar el alcance del Sprint agregando nuevos PBIs y reducirlo en el caso de retrasarnos.

Sprint Planning Meeting (Planificación de Sprint)

Al comienzo de cada Sprint se realiza una reunión de planificación del Sprint donde serán generados los acuerdos y compromisos entre el equipo de desarrollo y el Product Owner sobre el alcance del Sprint.

Esta reunión de planificación habitualmente se divide en dos partes con finalidades diferentes: una primera parte estratégica y enfocada en el "qué", y una segunda parte táctica cuyo hilo conductor principal es el "cómo".

Parte uno: ¿Qué trabajo será realizado?



Ilustración 17: Sprint Planning (Parte 1)

Podríamos decir que se trata de un taller donde el Product Owner expone todos y cada uno de los PBIs que podrían formar parte del Sprint, mientras que el equipo de desarrollo realiza todas las preguntas que crea necesarias para conocer sus detalles y así corroborar o ajustar sus estimaciones.

Aún asumiendo que los PBIs ya han sido estimados con anterioridad[16], debido al principio de "aceptar los cambios aun en etapas avanzadas del proyecto", es posible que en esta reunión aparezcan PBIs que no habían sido estimados anteriormente. Frente a esta situación, el equipo de desarrollo indagará y estimará esos PBIs de inmediato.

El objetivo buscado durante esta parte de la reunión es identificar "qué" es lo que el equipo de desarrollo va a realizar durante el Sprint, es decir, todos aquellos PBIs que el equipo se

16 La mecánica de esta estimación será explicada más adelante.

comprometerá a transformar en un producto funcionando y utilizable o en otras palabras: incremento funcional potencialmente entregable.

El Product Owner y el equipo de desarrollo deben participar de esta parte de la reunión como protagonistas principales. El ScrumMaster, al tiempo que facilita la reunión, también debe asegurar que cualquier *stakeholder* del proyecto que sea requerido para profundizar en detalles esté presente o sea contactado.

El equipo de desarrollo utiliza su capacidad productiva (también conocida como Velocidad o *Velocity*), obtenida de los Sprints pasados, para conocer hasta cuánto trabajo podría comprometerse a realizar. Esto determinaría en un principio cuáles serían los PBIs comprometidos en este Sprint.

Como se ha visto, hemos hablado en potencial*:* el equipo de desarrollo "podría", esto "determinaría". La razón es que cada uno de los ítems del Product Backlog debe ser discutido para entender cuáles son sus criterios de aceptación y así conocer en detalle qué se esperará de cada uno. De esta manera, el equipo de desarrollo discutirá con el Product Owner sobre cada PBI y generará un compromiso de entrega para aquellos que considera suficientemente claros como para comenzar a trabajar y que además podrían formar parte del alcance del Sprint que está comenzado. A esto se lo conoce como planificación basada en compromisos o Commitment-based Planning.

Al finalizar esta primera parte de la reunión, tanto el Product Owner como los stakeholders involucrados (si los hubiese) se retirarán, dejando así al ScrumMaster y al equipo de desarrollo

para que den comienzo a la segunda parte de esta reunión, que se describe a continuación.

Parte dos: ¿Cómo será realizado el trabajo?



Ilustración 18: Sprint Planning (Parte 2)

Durante este espacio de tiempo el equipo de desarrollo determinará la forma en la que llevará adelante el trabajo. Esto implica la definición inicial de un diseño de alto nivel, el cual será refinado durante el Sprint mismo y la identificación de las actividades que el equipo en su conjunto tendrá que llevar a cabo.

Se espera que el diseño sea emergente, es decir, que surja de la necesidad del equipo de desarrollo a medida que éste avance en el conocimiento del negocio. Por esta misma razón es que indicamos la no necesidad de realizar un diseño completo y acabado de lo que será realizado durante el Sprint. En cambio, se buscará un acuerdo de alto nivel que será bajado a detalle durante la ejecución de la iteración.

Esto mismo sucede con las actividades del Sprint, es decir que no es estrictamente necesario enumerar por completo todas las actividades que serán realizadas durante la iteración ya que muchas aparecerán a medida que avancemos. Recordemos que a esta altura los PBIs ya han sido estimados y el surgimiento de actividades durante el Sprint no habilita a incrementar las estimaciones de los PBIs, salvo excepciones donde la estimación inicial no había considerado la totalidad del esfuerzo necesario. Adicionalmente, es recomendable que las actividades duren idealmente menos de un día. Esto permitirá detectar bloqueos o retrasos durante las reuniones diarias (ver siguiente).

Si bien el Product Owner no participa de esta reunión, debería ser contactado en el caso de que el equipo de desarrollo necesite respuestas a nuevas preguntas con la finalidad de clarificar su entendimiento de las necesidades.

Al finalizar esta reunión, el equipo habrá arribado a un Sprint Backlog o Commited Backlog que representa el alcance del Sprint en cuestión. Este Sprint Backlog es el que se coloca en el taskboard (pizarra de actividades) del equipo. Se dará comienzo al desarrollo del producto para este Sprint.

Scrum Diario

Uno de los beneficios de Scrum está dado por el **incremento de la comunicación** dentro del equipo de proyecto. Esto facilita la coordinación de acciones entre los miembros del equipo de desarrollo y el conocimiento "en vivo" de las dependencias de las actividades que realizan.

Por otro lado, se requiere además **aumentar y explicitar los compromisos** asumidos entre los miembros del equipo de

desarrollo y **dar visibilidad a los impedimentos** que surjan del trabajo que está siendo realizando y que muchas veces nos impiden lograr los objetivos.

Estos tres objetivos: 1) incrementar la comunicación 2) explicitar los compromisos y 3) dar visibilidad a los impedimentos, son logrados mediante las reuniones diarias de Scrum (*Daily Scrums*). Estas reuniones tienen, como su nombre lo indica, una frecuencia diaria y no deberían llevar más de 15 minutos. Estos 15 minutos son un timebox, es decir, que no se pueden superar.

A la reunión diaria acude el ScrumMaster y el equipo de trabajo. En el caso de que sea necesario, se podrá requerir la presencia del Product Owner y de los stakeholders. De todas maneras, se intenta que sea una reunión abierta donde cualquier interesado en escuchar lo que sucede pueda participar en calidad de observador. Se recomienda que los observadores no participen activamente en la reunión, y mucho menos, que soliciten a los miembros del equipo justificación del progreso y explicación de los problemas.

Esta reunión es facilitada por el ScrumMaster. Todos y cada uno de los miembros toman turnos para responder las siguientes tres preguntas, y de esa manera comunicarse entre ellos:

1. ¿Qué hice desde la última reunión diaria hasta ahora?
2. ¿En qué voy a estar trabajando desde ahora hasta la próxima reunión diaria?
3. ¿Qué problemas o impedimentos tengo?

Es importante destacar que en ningún momento se trata de una reunión de reporte de avance o status al ScrumMaster ni a otras

personas. Por el contrario, es un espacio de estricta comunicación entre los miembros del equipo de desarrollo.

El objetivo de la primera pregunta (¿qué hice...?) es verificar el cumplimiento de los compromisos contraídos por los miembros del equipo en función del cumplimiento del objetivo del Sprint. La finalidad de la segunda pregunta (¿qué voy a hacer...?) es generar nuevos compromisos hacia el futuro. Cuando hablamos de compromisos, hacemos referencia a aquéllos que los miembros del equipo asumen ante sus compañeros.



Ilustración 19: Scrum Diario

La última pregunta (¿qué problemas...?) apunta a detectar y dar visibilidad a los impedimentos. Estos impedimentos no se resuelven en esta reunión, sino en posteriores. Es responsabilidad del ScrumMaster que se resuelvan lo antes posible, generando las reuniones que sean necesarias e involucrando a las personas correctas.

En el caso de que los PBIs del Sprint se hubiesen podido dividir en actividades de menos de un día: si una de estas actividades se

encuentra en progreso durante dos reuniones diarias seguidas (con 24hs de separación) claramente se advierte un retraso.

Revisión de Sprint

Al finalizar cada Sprint se realiza una reunión de revisión del Sprint (*Sprint Review*), donde se evalúa el incremento funcional potencialmente entregable construido por el equipo de desarrollo (el "qué"). En esta reunión el Equipo Scrum y los Stakeholders revisan el resultado del Sprint. Cuando decimos "resultado" hablamos de "producto utilizable" y "potencialmente entregable" que el los interesados utilizan y evalúan durante esta misma reunión, aceptando o rechazando así las funcionalidades construidas.



Ilustración 20: Revisión del Sprint

Los *Stakeholders* evalúan el producto construido y proveen feedback. Este feedback puede ser acerca de cambios en la funcionalidad construida o bien nuevas funcionalidades que surjan al ver el producto en acción.

Toda la retroalimentación que los stakeholders aporten debe ser ingresada como PBIs en el Product Backlog. Para esto, los PBIs

nuevos deben ser priorizados con respecto a todos los ya existentes en el Product Backlog. También es necesario que estos nuevos PBIs sean estimados antes de incluirlos como parte del Product Backlog ya que el Product Owner deberá decidir cuáles de los PBIs existentes cuya estimación de costo es similar a la de los nuevos PBIs deben ser eliminados para no incurrir en el incremento desmedido del alcance (Scope Creeping): si se agrega trabajo entonces debemos quitar trabajo de otro lado. El Product Owner cuenta para esto con la priorización de los ítems del Backlog como herramienta para la toma de este tipo de decisiones.

En el caso de que una funcionalidad sea rechazada, el PBI correspondiente reingresa al Product Backlog con máxima prioridad, para ser tratado en el siguiente Sprint. La única excepción a esta regla es que el Product Owner, por decisión propia, prefiera dar mayor prioridad a otros. En este caso, nada debe salir del Backlog ya que esto no sería considerado como un incremento en el alcance.

Al finalizar la revisión del producto, es recomendable definir la fecha de la próxima reunión de revisión, que corresponderá al final del Sprint siguiente. De este modo ya se tendrán las agendas bloqueadas a tal fin.

Retrospectiva

En un método empírico como Scrum, la retrospección del equipo es el corazón de la mejora continua y las prácticas emergentes. Mediante el mecanismo de retrospección, el equipo reflexiona sobre la forma en la que realizó su trabajo y los acontecimientos que sucedieron en el Sprint que acaba de

concluir para mejorar sus prácticas. Todo esto sucede durante la reunión de retrospectiva.

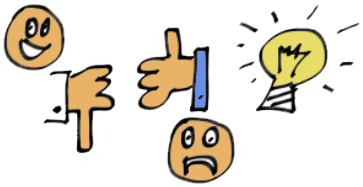


Ilustración 21: Retrospectiva

Esta reunión tiene lugar inmediatamente después de la reunión de revisión. Mientras que la reunión de revisión se destina a revisar el producto (el "qué"), la retrospectiva se centra en el proceso (el "cómo").

Este tipo de actividad necesita un ambiente seguro donde el Equipo Scrum pueda expresarse libremente, sin censura ni temores, por lo cual se restringe solo al Equipo de Desarrollo y al ScrumMaster. Personalmente yo también recomiendo la participación del Product Owner. En el caso de que se requiera la participación de stakeholders o gerentes, estos podrán ser convocados.

Valiéndose de técnicas de facilitación y análisis de causas raíces, se buscan tanto fortalezas como oportunidades de mejora. Luego, el Equipo Scrum decide por consenso cuáles serán las acciones de mejora a llevar a cabo en el siguiente Sprint. Estas acciones y sus impactos se revisarán en la próxima reunión de retrospectiva.

Refinamiento del Product Backlog

El refinamiento del Backlog es una actividad constante a lo largo de todo el Sprint, aunque algunos equipo prefieren concentrarla en una reunión que se realiza durante el Sprint y en función de las necesidades. Su objetivo es profundizar en el entendimiento de los PBIs que se encuentran más allá del Sprint actual y así dividirlos en PBIs más pequeños, si lo requieren, y estimarlos. Idealmente se revisan y detallan aquellos que potencialmente se encuentren involucrados en los próximos dos o tres Sprints.

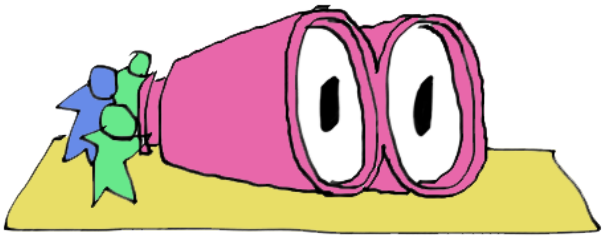


Ilustración 22: Refinamiento del Product Backlog

Otro objetivo importante que se debe perseguir en esta reunión es la detección de riesgos implícitos en los PBIs que se estén analizando, y en función de ellos revisar y ajustar las prioridades del Product Backlog.

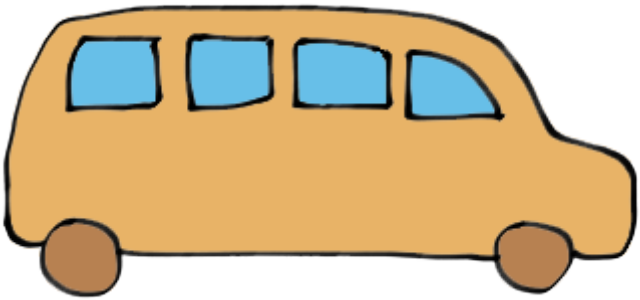
La participación de todo el Equipo Scrum es esencial para el éxito de esta reunión. Sin ellos, esta actividad, no tendría sentido.

En el caso de que se realice como una reunión, la responsabilidad de convocarla es del Product Owner, entre una y dos veces por Sprint, facilitadas por el ScrumMaster.

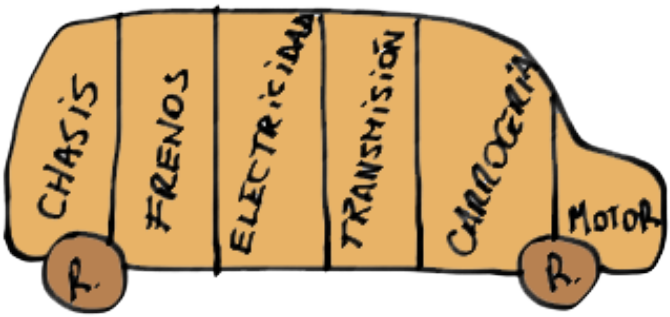
3. Desarrollo Evolutivo

Creación Evolutiva

Supongamos que nos han contratado de una empresa de transporte para construir autobuses para el traslado de niños desde su casa a la escuela y desde la escuela a su casa.



Luego de analizar las características o funcionalidades que el autobús debe tener, hemos dividido la problemática en:



Una alternativa para construir el autobús sería dedicar la primera entrega al chasis y los frenos, la segunda al motor y la

carrocería, la tercera a la transmisión, etc., tal como se muestra a continuación.

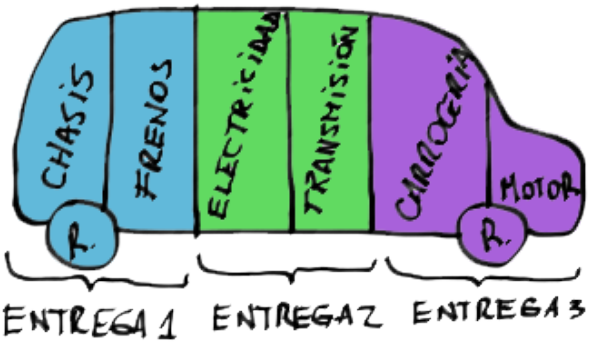


Ilustración 23: Desarrollo Secuencial

Sin embargo, si nosotros decidimos construir el vehículo de forma evolutiva e incremental deberíamos tener una unidad funcionando al final de cada iteración, lo que significa segmentar el desarrollo de forma transversal a dichas funcionalidades con el fin de proveer una pequeña porción de cada una en cada entrega, formando un producto utilizable:

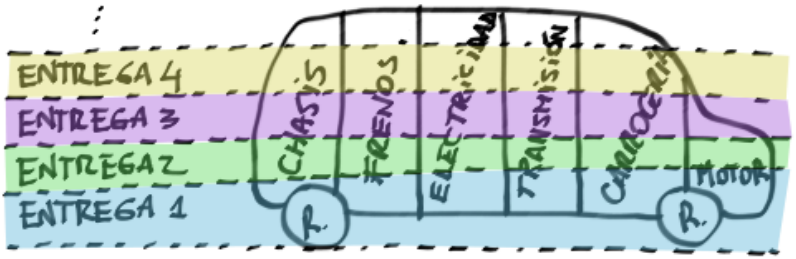


Ilustración 24: Desarrollo Evolutivo

De esta forma, podríamos buscar los siguientes objetivos:

- Entrega 1: base del chasis, ruedas de motocicleta, dirección básica. Objetivo: verificar rodamiento, traslado seguro y maniobrabilidad
- Entrega 2: sumamos refuerzos del chasis, transmisión básica, suspensores, ruedas de segunda mano, motor básico, paredes de la carrocería. Objetivo: verificar dimensiones y suspensión.
- Entrega 3: electricidad, motor diesel, frenos, techo, asientos, ruedas nuevas, sistema ABS. Objetivo: Habilidad de circular públicamente.
- Entrega 4: etc.
- Entrega 5: etc.

Partiendo de esta base, vamos a introducir tres conceptos complementarios entre sí: ***Minimum Viable Product***, ***Minimum Marketable Feature*** y ***Visual Story Mapping***.

Minimum Viable Product

El Minimum Viable Product (MVP) es la versión mínima de un producto, tal que nos permita recolectar la mayor cantidad de información de nuestro mercado y clientes con el menor esfuerzo posible. Consiste en hacer foco en las características mínimas y necesarias para que el producto pueda lanzarse al mercado. Esto nos permitirá:

- Evitar crear productos que nadie necesita
- Maximizar el aprendizaje por dólar invertido

El MVP es una estrategia de *Lean Startup* que apunta a acercarnos a nuestros clientes con la menor inversión

posible (tiempo/dinero) y con ello determinar si nuestro producto es o no es viable.

Minimum Marketable Features

Todas las metodologías ágiles coinciden en que un producto debe construirse de forma evolutiva en pequeñas entregas. De todas formas no es suficiente, como vimos anteriormente, dividir el producto en tres o cuatro entregas sucesivas, sino que debemos hacerlo de forma criteriosa para que cada entrega pueda aportar valor suficiente a los usuarios finales. Esos grupos de características se denominan MMF: Minimum Marketable Features, y pueden definirse como "*el conjunto más pequeño posible de funcionalidad que, por si misma, tiene valor en el mercado*"[17]

Visual Story Mapping

Conjugando el Desarrollo Evolutivo, la Priorización del Backlog y el concepto de Minimum Marketable Feature, Jeff Patton plantea una técnica de Análisis Ágil llamada **Mapeo Visual de Historias** o *Visual Story Mapping*[18].

La teoría del Visual Story Mapping comienza en un nivel "humano" identificando los **Objetivos** que toda persona persigue y dividiéndolos en **Actividades** para las cuales deben utilizarse **Herramientas,** resultando entonces en una jerarquía de Objetivos → Actividades → Herramientas.

17 "Phased Releases" , James Shore, 2004
18 Visual Story Mapping, Jeff Patton, 2009

El último nivel denominado “herramientas”, puede desagregarse a su vez en diferentes niveles de confort. Por este mismo principio una persona puede viajar de una ciudad a otra en un automóvil del año 1965 o en un último modelo siendo que la actividad “llegar de una ciudad a otra” seguirá cumpliéndose.

Este nivel de confort está dado por 1) la necesidad de negocio (necesito que el viaje se haga en menos de 30 minutos) y 2) cuánto estemos dispuestos a invertir (precio del auto).

Haciendo una analogía con las organizaciones, esta jerarquía de Objetivo → Actividad → Herramienta puede traducirse en Proceso de Negocio → Actividad → Software.

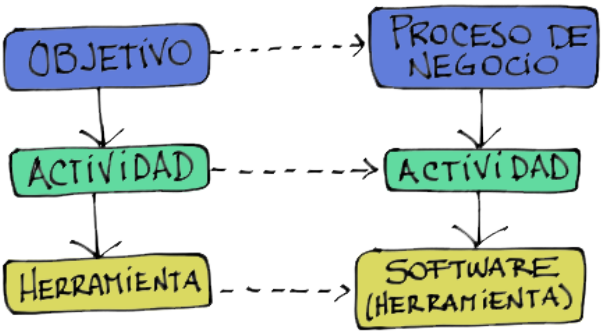


Ilustración 25: Jerarquía operativa

El Software como herramienta también puede otorgarnos diferentes niveles de confort. Uniendo entonces el concepto de MMF y de nivel de confort, deberíamos pensar la construcción del software de forma evolutiva, naciendo desde lo mínimo posible (MMF) e ir escalando en los niveles de confort de las funcionalidades iteración tras iteración, tratando de abarcar tanta

funcionalidad como sea posible en la extensión del proceso de negocio y no tanto en profundidad.

Teniendo en cuenta entonces que el software será construido evolutivamente, incrementando la funcionalidad entrega tras entrega y sumando elementos visuales, surge entonces una herramienta colaborativa para analizar el alcance del software a ser construido y para dividirlo en diferentes entregas. Esta técnica visual utiliza elementos físicos como marcadores, notas autoadhesivas y papel afiche con el propósito de fomentar la colaboración entre las personas.

Proceso de Análisis Ágil

Para el desarrollo de esta técnica en forma práctica, nos basaremos en un ejemplo real: el análisis de un sistema de gestión de cursos de capacitación. Este ejemplo servirá como columna vertebral en la que vamos a ir aplicando las técnicas ágiles descriptas.

Roles de Usuario

Previo al análisis del sistema, es necesario identificar los posibles usuarios que tendrá. Para esto utilizaremos una técnica colaborativa descripta por Mike Cohn[19] basada en el trabajo de Constantine & Lockwood[20]. Esta técnica se realiza en equipo, durante un taller donde el cliente y tantos desarrolladores como sea posible colaboran en la identificación de los roles. El taller se compone de cuatro actividades:

19 Agile Estimating and Planning, Mike Cohn, 2005
20 Software for Use, Constantine & Lockwood, 1999

1. *Brainstorming* de un conjunto inicial de roles
2. Organización del conjunto inicial de roles
3. Consolidación de roles
4. Refinamiento de roles

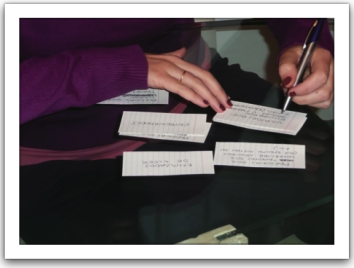
Brainstorming de un conjunto inicial de roles

Como se ha mencionado anteriormente, la intención de esta actividad es que sea lo más colaborativa posible. Tanto el cliente como el Equipo completo deberían participar, aunque muchas veces será suficiente con la participación de un conjunto representativo del Equipo de desarrollo.

La reunión se lleva a cabo sobre una mesa lo suficientemente grande para todos los participantes. Cada uno toma varias fichas de una pila dispuesta en el centro de la mesa y escribe un rol en la misma. Siendo que esta actividad es un *Brainstorming* no debe haber discusión ni censura para cada rol que alguien escribe.

Una opción que suele funcionar muy bien es que los participantes anoten tantos roles como sea posible en sus fichas, en silencio y sin compartirlos con el resto de las personas.

En nuestro caso, los involucrados identificaron varios roles cada uno, como se muestra en las fotografías.







Organización del conjunto inicial de roles

Una vez que el grupo haya terminado de identificar los roles, el próximo paso es organizarlos. Para esto, los dispondrá sobre la mesa de forma tal que las similitudes queden representadas de forma visual. Esto se logra solapando levemente aquellos roles que tienen pocas similitudes, solapando por completo aquellos que son iguales y separando los que no tienen relación.

Para poder llegar a ese resultado, los participantes deben compartir los roles con el resto del equipo y describir cada uno de ellos, discutiendo e indagando para poder entender las similitudes y diferencias. Esto a su vez ayudará a diseminar el conocimiento entre los integrantes del Equipo. Para nuestro sistema bajo análisis, el resultado de este ejercicio fue como se muestra en las siguientes fotografías:

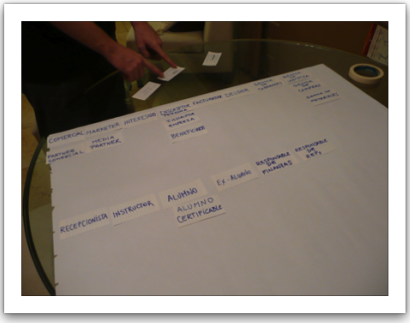




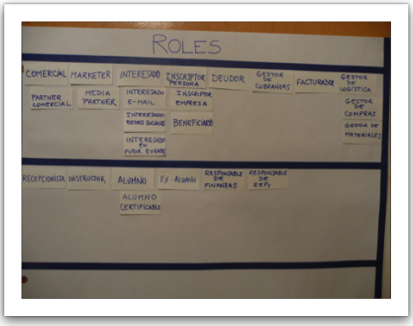
Consolidación de roles

Luego de haber agrupado los roles, el siguiente paso será consolidar y condensar. Para esto se comienza por aquellas fichas que tienen el mayor solapamiento, se discuten para entender si podrían condensarse en un único rol y, en el caso de que sea posible hacerlo, se buscará un único nombre para que las represente. En nuestro ejemplo, luego de esta dinámica se llegó al siguiente resultado:





Luego de discutir los diferentes roles, se agruparon de forma tal de representar los roles principales en los niveles superiores, y los sub-roles o especializaciones en los niveles inferiores, trasladados a su vez, hacia la derecha:



Refinamiento de roles

El cuarto y último paso de la identificación de roles consiste en lograr su refinamiento mediante la descripción de las siguientes características:

- Frecuencia de uso del sistema por parte del usuario
- Nivel de experiencia del usuario en el dominio del problema
- El nivel general de experiencia del usuario con el uso de computadoras

El nivel general de experiencia del usuario con el sistema

Objetivo del usuario con la utilización del sistema

Descripción refinada de los roles:

- **Comercial**
  Uso intensivo del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la de proveer información sobre los diferentes cursos frente a las consultas de los interesados. Esto incluye programa, contenidos, fechas, precios y cantidad de vacantes. También debe conocer el estado de completitud de cada curso en el calendario y tener la posibilidad de crear nuevos eventos.

  - **Partner Comercial**
    Ídem Comercial, con la particularidad que los eventos creados por un Partner Comercial se registran como "tentativos" hasta que un Comercial los confirma.

- **Marketer**
  Uso frecuente del sistema con conocimiento limitado del

dominio del problema. Posee un nivel avanzado de experiencia en la utilización de computadoras y nivel intermedio de experiencia con el uso del sistema en particular y alto nivel de experiencia en el uso de redes sociales como Twitter y Facebook. Su responsabilidad será la de promover y difundir los eventos en internet.

- **Media Partner**
  Uso eventual del sistema con bajo conocimiento del dominio del problema. Posee un nivel avanzado de experiencia en la utilización de computadoras y nivel bajo de experiencia con el uso del sistema en particular y alto nivel de experiencia en su sitio web. Su responsabilidad será la de difundir los eventos entre los usuarios de sus sitios web, realizar sorteos y proveer códigos de descuento. También debe conocer el estado de su cuenta en el caso de obtener beneficios económicos en base a referidos.

- **Interesado**
  Uso infrecuente del sistema sin conocimiento del dominio del problema. Se asumirá un nivel avanzado de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Su interés será consultar el calendario y contenidos de los eventos.

  - **Interesado E-mail**
    Ídem interesado, pero su interés es recibir información vía e-mail.

  - **Interesado Redes Sociales**

Ídem interesado, pero su interés es recibir información vía redes sociales.

- **Interesado en Futuros Eventos**
  Un tipo particular de Interesado, cuyo foco está en futuros eventos en una ciudad o país en particular.

- **Persona a Inscribirse**
  Uso infrecuente del sistema sin conocimiento del dominio del problema. Se asumirá un nivel avanzado de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Su interés es inscribirse a un determinado evento.

  - **Empresa con Personas a Inscribir**
    Uso infrecuente del sistema sin conocimiento del dominio del problema. Se asumirá un nivel intermedio de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Su interés es inscribirse a un determinado grupo de personas, todas de una misma empresa a un evento en particular.

  - **Beneficiario de Empresa**
    Uso infrecuente del sistema sin conocimiento del dominio del problema. Se asumirá un nivel avanzado de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Su interés es recibir información sobre los eventos a los que fue inscripto por una tercera persona.

- **Deudor**
  Uso infrecuente del sistema sin conocimiento del dominio del problema. Se asumirá un nivel avanzado de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Su interés es la realización de los pagos pendientes para poder asistir al evento al cual está inscripto.

- **Gestor de Cobranzas**
  Uso intensivo del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será el seguimiento de los pagos de los diferentes eventos.
- **Facturador**
  Uso intensivo del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la realización de las facturas a individuos u organizaciones.
- **Gestor de Logística**
  Uso periódico del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será el seguimiento de todo lo que hace a la logística de un determinado evento.
  - **Gestor de Compras**
    Uso periódico del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de

experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la realización de todas las compras necesarias para los eventos.

- **Gestor de Materiales**
  Uso periódico del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la determinación y administración de los materiales y cantidades para cada tipo de evento.

- **Recepcionista**
  Uso frecuente del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la recepción de los asistentes, la toma de asistencia y la autorización de participación a los mismos.
- **Instructor**
  Uso frecuente del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio a avanzado de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su objetivo será la creación de tipos de eventos y la provisión de los contenidos, programas, lecturas y material asociado a cada uno de ellos. También será su responsabilidad la evaluación de los exámenes rendidos por los alumnos.
- **Alumno**
  Uso infrecuente del sistema sin conocimiento del

dominio del problema. Se asumirá un nivel avanzado de experiencia en la utilización de computadoras y bajo nivel de experiencia con el uso del sistema en particular. Está interesado en acceder a los contenidos de los diferentes cursos o eventos a los cuales asiste, como así también en poder rendir los exámenes que cada curso requiera y obtener los correspondientes certificados de examen y asistencia.

  - **Alumno Certificable**
    Ídem Alumno. Su interés consiste en poder recibir las instrucciones necesarias para solicitar la certificación correspondiente. Las certificaciones generalmente están relacionadas a la completitud de una serie determinada de cursos o un curso en particular.

- **Ex-Alumno**
  Ídem Alumno. Su interés es recibir información sobre nuevos eventos o cursos relacionados o correlativos a los cursos o eventos a los que ha asistido.
- **Responsable de Finanzas**
  Uso intensivo del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la planificación y elaboración de presupuestos para cursos y eventos y el conocimiento de los resultados económicos de los mismos.
- **Responsable de REPs**
  Uso intensivo del sistema con gran conocimiento del dominio del problema. Posee un nivel intermedio de experiencia en la utilización de computadoras y alto nivel de experiencia con el uso del sistema en particular. Su responsabilidad será la publicación de los alumnos en

los cursos declarados en los sistemas de las organizaciones de las cuales la empresa es REP (Registered Education Provider).

Visual Story Mapping en la Práctica

Identificación de los Procesos de Negocio

A continuación se realizará una identificación de procesos de negocio que el sistema deberá resolver, independientemente de los roles encontrados en el ejercicio anterior.

Los procesos de negocio identificados como parte de este taller fueron:

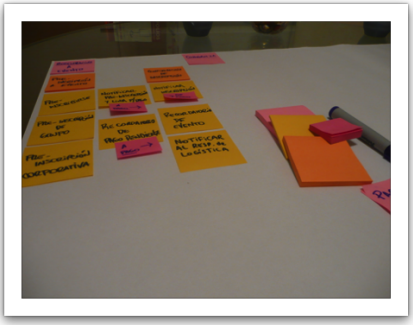
- Venta de Evento
- Registración a Evento
- Cobranza de Evento
- Facturación de Evento
- Evaluación de Evento
- Logística de Evento

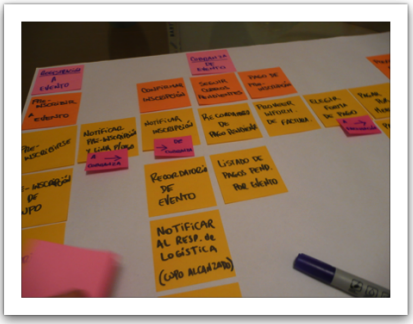
Identificación de Funcionalidades del Software (Herramientas)

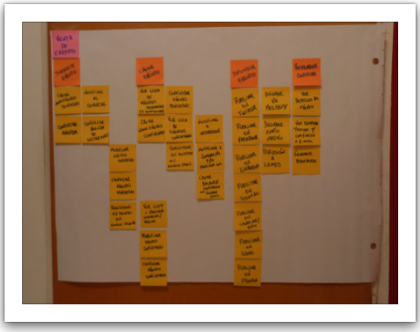
Continuando con la práctica de Visual Story Mapping, el próximo paso consiste en la identificación de las funcionalidades con las que el sistema deberá contar. Esta actividad la realizamos teniendo en cuenta todos los roles identificados, efectuando sucesivas "pasadas" por todos los procesos de negocio y evaluando que cada uno de los roles involucrados en ellos cuenten con las funcionalidades requeridas para la realización de sus objetivos. Al igual que la identificación de

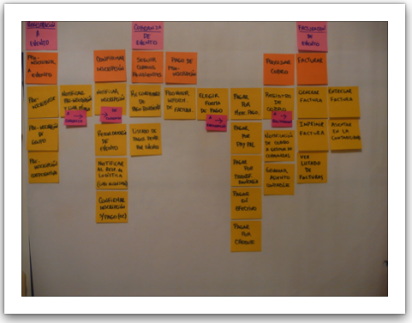
roles, esta actividad se realiza en forma colaborativa junto al Product Owner y la mayor cantidad de miembros del equipo posible. En las fotografías siguientes se podrán identificar los procesos de negocio en color rosa, las actividades en color naranja y las funcionalidades en color amarillo:






NOTIFICAR AL RESP. de LOGÍSTICA
(CUPO ALCANZADO)





Para el sistema en cuestión hemos identificado las siguientes funcionalidades por cada uno de los procesos de negocio:

- Venta de Evento
  - Sugerir Evento
    - Crear evento tentativo
    - Notificar a comercial
    - Consultar agenda de eventos
    - Consultar agenda de instructores
    - Modificar evento tentativo
    - Cancelar evento tentativo
    - Registrar evento tentativo en Google Calendar
  - Crear Evento
    - Ver listado de eventos tentativos
    - Confirmar evento tentativo
    - Crear un evento confirmado
    - Ver listado de eventos confirmados
    - Registrar evento confirmado en Google Calendar
    - Notificar a Instructor sobre la confirmación de evento
    - Notificar a Comercial o Partner Comercial sobre la confirmación de evento
    - Crear balance contable del evento en Google Docs
    - Ver listado de eventos tentativos agrupados por Partner Comercial y/o Región
    - Modificar evento confirmado
    - Cancelar evento confirmado
  - Difundir Evento
    - Publicar Evento en
      - Twitter
      - Facebook
      - LinkedIn
    - Listar evento en sitio web
    - Difundir vía Mailchimp
    - Difundir en forma masiva
    - Difundir a leads comerciales
  - Responder Consultas
    - Publicar detalles de evento
    - Generar texto con fechas y valores para “Copy & Paste”
    - Generar brochure del evento
  - Visualizar Estado de Inscripciones
    - Dashboard de inscripciones a cursos
- Registración a Evento
  - Pre-Inscripción a Evento
    - Pre-Inscripción individual
    - Pre-Inscripción de grupo
    - Pre-Inscripción corporativa
    - Notificación de Pre-Inscripción y pasos siguientes

- Confirmación de Inscripción
  - Notificar Inscripción
  - Recordatorio de Evento
  - Notificación al responsable logístico sobre cupo alcanzado
  - Confirmar inscripción sin pago (pago a cuenta)
- Cobranza de Evento
  - Seguimiento de Cobros Pendientes
    - Recordatorio de pago pendiente
    - Listado de Pagos Pendientes por evento
  - Pago de Pre-inscripción
    - Obtención de información de facturación
    - Elección de forma de pago
    - Pagar en/por
      - Efectivo
      - Cheque
      - Transferencia Bancaria
      - PayPal
      - Mercado Pago
  - Procesar Cobro
    - Registro de Pago
    - Notificación de cobro a Gestor de Finanzas
    - Generación de asiento contable
- Facturación de Evento
  - Facturar Evento
    - Generar Factura
    - Imprimir Factura
    - Ver listado de Facturas
    - Entregar Factura
    - Asentar Factura en Contabilidad
- Evaluación de Evento
  - Responder Preguntas
    - Responder Preguntas Multiple-Choice
    - Responder Preguntas a Desarrollar
    - Finalización de Evaluación (Pantalla)
    - Notificación de Finalización de Evaluación (E-mail)
  - Corregir Evaluación
    - Corrección automática de preguntas multiple-choice
    - Listado de evaluaciones a corregir
    - Corrección manual de preguntas a desarrollar
    - Feedback de corrección
  - Notificar Resultado
    - Notificación del resultado por e-mail
    - Generación de certificado de evaluación aprobada
  - Recuperar Evaluación
    - Listado de recuperatorios pendientes
    - Listado de preguntas erradas
    - Responder preguntas erradas
    - Recalculo automático de resultado
    - Corrección manual de preguntas a desarrollar
    - Feedback de corrección

- Logística de Evento
  - Gestionar Maestros de Logística
    - ABM Tipos de Eventos
    - ABM Checklist Template
    - ABM Materiales
  - Generar Checklist
    - Instanciación de Checklist de Evento
  - Actualizar Checklist
    - Actualización de datos de Checklist
    - Notificar a responsable de logística
  - Operar Checklist
    - Listado de eventos con progreso de checklist
    - Detalle de checklist de evento

Identificación de MVP y posteriores entregas

Como hemos indicado anteriormente[21], la construcción del sistema se realizará en forma orgánica o evolutiva, naciendo desde el MVP (producto mínimo viable) y produciendo incrementos funcionales (MMFs) potencialmente entregables en cada iteración.

MVP (o Entrega 1) – Objetivo: Comercializar Eventos

- Venta de Evento
  - Crear Evento
    - Crear un evento confirmado
    - Ver listado de eventos confirmados
    - Modificar evento confirmado
    - Cancelar evento confirmado
  - Difundir Evento
    - Listar evento en sitio web
  - Responder Consultas
    - Publicar detalles de evento
    - Generar texto con fechas y valores para “Copy & Paste” en e-mail de respuesta
  - Visualizar Estado de Inscripciones
    - Dashboard de inscripciones a cursos
- Registración a Evento

21 Ver MVP, MMF y Visual Story Mapping

- Pre-Inscripción a Evento
  - Pre-Inscripción individual
- Confirmación de Inscripción
  - Notificar Inscripción
  - Confirmar inscripción sin pago (pago a cuenta)

- Cobranza de Evento
  - Seguimiento de Cobros Pendientes
    - Listado de Pagos Pendientes por evento
  - Pago de Pre-inscripción
    - Pagar en/por
      - Efectivo
      - Cheque
      - Transferencia Bancaria
  - Procesar Cobro
    - Registro de Pago
    - Notificación de cobro a Gestor de Finanzas

Entrega 2 – Objetivo: Toma de evaluaciones on-line

- Evaluación de Evento
  - Responder Preguntas
    - Responder Preguntas Multiple-Choice
    - Finalización de Evaluación (Pantalla)
    - Notificación de Finalización de Evaluación (E-mail)
  - Corregir Evaluación
    - Corrección automática de preguntas multiple-choice
  - Notificar Resultado
    - Notificación del resultado por e-mail
    - Generación de certificado de evaluación aprobada
  - Recuperar Evaluación
    - Listado de recuperatorios pendientes
    - Listado de preguntas erradas
    - Responder preguntas erradas
    - Recálculo automático de resultado

Entrega 3 – Objetivo: Pre-Inscripción Individual y Corrección de Exámenes a Desarrollar

- Registración a Evento
  - Pre-Inscripción a Evento
    - Notificación de Pre-Inscripción y pasos siguientes
- Evaluación de Evento
  - Responder Preguntas
    - Responder Preguntas a Desarrollar
  - Corregir Evaluación
    - Listado de evaluaciones a corregir
    - Corrección manual de preguntas a desarrollar
    - Feedback de corrección

Entrega 4 – Objetivo: Pre-Inscripción Corporativa y Seguimiento de Pagos

- Registración a Evento
  - Pre-Inscripción a Evento
    - Pre-Inscripción corporativa
  - Confirmación de Inscripción
    - Recordatorio de Evento
    - Notificación al responsable logístico sobre cupo alcanzado
- Cobranza de Evento
  - Seguimiento de Cobros Pendientes
    - Recordatorio de pago pendiente
  - Pago de Pre-inscripción
    - Obtención de información de facturación
    - Elección de forma de pago
    - Pagar en/por
      - PayPal
      - Mercado Pago

Entrega 5 – Objetivo: Logística de Eventos

- Logística de Evento
  - Gestionar Maestros de Logística
    - ABM Tipos de Eventos
    - ABM Checklist Template
    - ABM Materiales
  - Generar Checklist
    - Instanciación de Checklist de Evento
  - Actualizar Checklist
    - Actualización de datos de Checklist
    - Notificar a responsable de logística
  - Operar Checklist
    - Listado de eventos con progreso de checklist
    - Detalle de checklist de evento

Entrega 6 – Objetivo: Eventos Tentativos

- Venta de Evento
  - Sugerir Evento
    - Crear evento tentativo
    - Notificar a comercial
    - Consultar agenda de eventos
    - Modificar evento tentativo
    - Cancelar evento tentativo
  - Crear Evento
    - Ver listado de eventos tentativos
    - Confirmar evento tentativo
    - Notificar a Comercial o Partner Comercial sobre la confirmación de evento
    - Notificar a Instructor sobre la confirmación de evento
    - Ver listado de eventos tentativos agrupados por Partner Comercial y/o Región
  - Responder Consultas
    - Generar brochure del evento
- Registración a Evento
  - Pre-Inscripción a Evento
    - Pre-Inscripción de grupo

Entrega 7 – Objetivo: Integración con sistemas externos

- Venta de Evento
  - Sugerir Evento
    - Consultar agenda de instructores
    - Registrar evento tentativo en Google Calendar
  - Crear Evento
    - Registrar evento confirmado en Google Calendar
    - Crear balance contable del evento en Google Docs
  - Difundir Evento
    - Publicar Evento en
      - Twitter
      - Facebook
      - LinkedIn
    - Difundir vía Mailchimp
    - Difundir en forma masiva
    - Difundir a leads comerciales
- Cobranza de Evento
  - Procesar Cobro
    - Generación de asiento contable
- Facturación de Evento
  - Facturar Evento
    - Generar Factura
    - Imprimir Factura
    - Ver listado de Facturas
    - Entregar Factura
    - Asentar Factura en Contabilidad

4. Historias de Usuario

Valor: Software funcionando por sobre la documentación extensiva

Principio: El método más eficiente y eficaz de transmitir información hacia y dentro de un equipo de desarrollo es mediante la comunicación cara a cara.

Agile Manifesto – 2001

Las **Historias de Usuario** surgieron en *eXtremme Programming (XP)* como una respuesta a una situación habitual en los proyectos de desarrollo do software: los clientes o especialistas de negocio se comunican con los equipos de desarrollo a través de extensos documentos conocidos como especificaciones funcionales. A su vez, las especificaciones funcionales son la documentación de supuestos y están sujetas a interpretaciones, lo que causa malos entendidos y que finalmente el software construido no se corresponda con la realidad esperada.

Una de las principales razones por las cuales la utilización de especificaciones detalladas como medio de comunicación no conduce a resultados satisfactorios es porque solo cubre una porción mínima (7%) del espectro de la comunicación humana: el contenido. Según Albert Mehrabian, la comunicación humana se compone de tres partes[22]:

1. En un 7%: El contenido (las palabras, lo dicho)
2. En un 38%: El tono de la voz

22 "Silent messages: Implicit communication of emotions and attitudes.", Albert Mehrabian, 1981

3. En un 55%: Las expresiones faciales

Por esto se concluye que para tener una comunicación sólida, completa, es necesario el contacto cara-a-cara entre los interlocutores. En un esfuerzo orientado a que esas conversaciones existan, podemos decir que las Historias de Usuario son especificaciones funcionales que invitan a la conversación para que el detalle sea consecuencia de esta última y no un remplazo.

Componentes de una Historia de Usuario

Una Historia de Usuario se compone de 3 elementos, también conocidos como "las tres Cs"[23] de las Historias de Usuario:

- **Card (Ficha)** – Toda historia de usuario debe poder describirse en una ficha de papel pequeña. Si una Historia de Usuario no puede describirse en ese tamaño, es una señal de que estamos traspasando las fronteras y comunicando demasiada información que debería compartirse cara a cara.
- **Conversación –** Toda historia de usuario debe tener una conversación con el Product Owner. Una comunicación cara a cara que intercambia no solo información sino también pensamientos, opiniones y sentimientos.
- **Confirmación** – Toda historia de usuario debe estar lo suficientemente explicada para que el equipo de desarrollo sepa qué es lo que debe construir y qué es lo que el Product Owner espera. Esto se conoce también como *Criterios de Aceptación*.

23 "Essential XP: Card, Conversation, Confirmation", Ron Jeffries, 2001



Ilustración 26: Componentes de una historia de usuario

Redacción de una Historia de Usuario

Mike Cohn sugiere una determinada forma de redactar Historias de Usuario bajo el siguiente formato:

Como (rol) Necesito (funcionalidad) Para (beneficio)[24]

24 "Advantages of the "As a user, I want" user story template", Mike Cohn, 2008

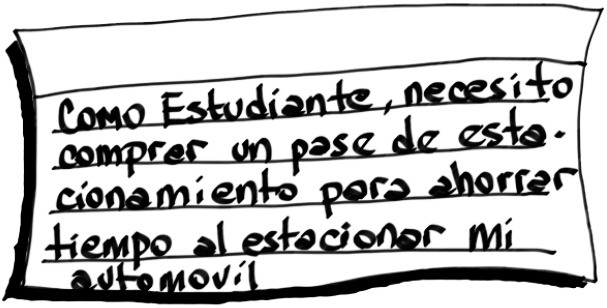


Ilustración 27: Redacción típica de una historia de usuario

Los beneficios de este tipo e redacción son, principalmente:

Primera Persona

La redacción en primera persona de la Historia de Usuario invita a quien la lee a ponerse en el lugar del usuario.

Priorización

Tener esta estructura para redactar la Historia de Usuario ayuda al Product Owner a priorizar. Si el Product Backlog es un conjunto de ítems como “Permitir crear un evento tentativo”, “Confirmar un evento tentativo”, “Notificar al responsable de logística”, “Ver el estado de inscripciones”, etc. el Product Owner debe trabajar más para comprender cuál es la funcionalidad, quien se beneficia y cuál es el valor de la misma.

Propósito

Conocer el propósito de una funcionalidad permite al equipo de desarrollo plantear alternativas que cumplan con el mismo propósito en el caso de que el costo de la funcionalidad solicitada sea alto o su construcción no sea viable.

INVEST - Características de una Historia de Usuario

Se recomienda que toda Historia de Usuario cumpla con 6 características que podemos recordar bajo la regla mnemotécnica "INVEST"[25]:

Independientes (I)

Las Historias de Usuario deben ser independientes de forma tal que no se superpongan en funcionalidades y que puedan planificarse y desarrollarse en cualquier orden.

Muchas veces esta característica no puede cumplirse para el 100% de las Historias. El objetivo que debemos perseguir es preguntarnos y cuestionarnos en cada Historia de Usuario si hemos hecho todo lo posible para que ésta sea independiente del resto.

Negociable (N)

Una buena Historia de Usuario es *Negociable*. No es un contrato explícito por el cual se debe entregar todo-o-nada. Por el contrario, el alcance de las Historias (sus criterios de aceptación) podrían ser variables: pueden incrementarse o eliminarse con el correr del desarrollo y en función del feedback del usuario y/o la performance del Equipo. En el caso de que uno o varios criterios de aceptación se eliminen de una Historia de Usuario, estos se transformarán en una o varias Historias de Usuario nuevas.

25 "INVEST in Good Stories, and SMART Tasks", Bill Wake, 2003

Esta es la herramienta que el Product Owner y el Equipo tienen para negociar el alcance de cada Sprint.

Valorable (V)

Una Historia de Usuario debe ser Valorable por el Product Owner. Los Desarrolladores pueden tener actividades técnicas como parte del BackLog, pero para que puedan ser consideradas una Historia de Usuario, deben ser enmarcadas de forma tal que el Product Owner las considere importantes, caso contrario, no deberían formar parte del BackLog.

En general, esta característica representa un desafío a la hora de dividir Historias de Usuario. Bill Wake propone pensar en una Historia de Usuario como si fuese una torta de múltiples capas, por ejemplo: una capa de persistencia, una capa de negocio, una capa de presentación, etc. Cuando dividamos esa Historia de Usuario, lo que vamos a estar sirviendo es una parte de esa "torta" y el objetivo debería ser darle al Product Owner la esencia de la "torta" completa, y la mejor manera de hacerlo es cortando una rodaja vertical de esta "torta" a través de todas las capas. Los Desarrolladores tenemos una inclinación especial de trabajar en una capa a la vez hasta completarla, pero una capa de persistencia de datos completa y terminada tiene muy poco o ningún valor para el Product Owner si no hay una capa de negocio y de presentación.

Estimable (E)

Una Historia de Usuario debería ser estimable. Mike Cohn[26], identifica tres razones principales por las cuales una Historia de Usuario no podría estimarse:

- **La Historia de Usuario es demasiado grande**. En este caso la solución sería dividir la Historia de Usuario en historias más pequeñas que sean estimables.
- **Falta de conocimiento funcional**. En este caso la Historia de Usuario vuelve al Product Owner para bajar en detalle la Historia o inclusive (y recomendable) tener una conversación con el Equipo de Desarrollo.
- **Falta de conocimiento técnico**. Muchas veces el Equipo de Desarrollo no tiene el conocimiento técnico suficiente para realizar la estimación. En estos casos el Equipo de Desarrollo puede dividir la historia en 1) un time-box conocido como "spike" que le permita investigar la solución y proveer una estimación más certera y 2) la funcionalidad a desarrollar como parte de la Historia en si misma.

Pequeña (Small)

Toda Historia de Usuario debe ser lo suficientemente pequeña de forma tal que permita ser estimada por el Equipo de Desarrollo. Algunos Equipos fijan el tamaño de una Historia de usuario como no más de dos semanas de una persona. Si bien no es una medida explícita, tener entre 4 y 6 Historias de Usuario por Sprint es una buena señal de tamaño.

---

26 "User Stories Applied", Mike Cohn, 2003

Las descripciones de las Historias de Usuario también deberían ser pequeñas, y escribirlas en fichas pequeñas ayuda a que eso suceda.

Verificable (Testable)

Una buena Historia de Usuario es Verificable. Se espera que el Product Owner no solo pueda describir la funcionalidad que necesita, sino que también logre verificarla (probarla). Algunos Equipos acostumbran solicitar los criterios de aceptación antes de desarrollar la Historia de Usuario. Si el Product Owner no sabe cómo verificar una Historia de Usuario o no puede enumerar los criterios de aceptación, esto podría ser una señal de que la Historia en cuestión no está siendo lo suficientemente clara.



Ilustración 28: Características de una buena historia de usuario

Definición de Listo

También conocido como *Definition of Ready*, es el conjunto de características que una Historia de Usuario debe cumplir para que el Equipo de Desarrollo pueda comprometerse a su entrega, es decir, incluirla en un Sprint Backlog. Una típica definición de listo podría ser:

- La Historia de Usuario debe ser INVEST
- Todos sus pre-requisitos están resueltos (ej: dependencias con otros Equipos)



Ilustración 29: Definición de Listo (*Definition of Ready*)

Definición de Terminado

También conocido como *Definition of Done*, es el conjunto de características que una Historia de Usuario debe cumplir para que el equipo de desarrollo pueda determinar si ha terminado de trabajar en ella. Un típico criterio de "Terminado" podría ser:

- Todos los criterios de aceptación funcionan correctamente
- Todos los archivos fuentes están en el repositorio de código fuente y el *build* se ejecutó exitosamente

- En el caso de Product Owners muy exigentes: El Product Owner dio su visto bueno de la funcionalidad construida antes de llegar a la Review Meeting.



Ilustración 30: Definición de Terminado (*Definition of Done*)

Las Historias de Usuario de Nuestro Producto

A continuación redactamos las Historias de Usuario que comprenden el Product Backlog de nuestro producto. Dada la naturaleza evolutiva del alcance de un proyecto ágil, las Historias de Usuario de mayor prioridad estarán más detalladas que las Historias de Usuario de menor prioridad, las cuales inclusive podrían considerarse EPICS (agrupaciones de varias Historias de usuario):

Entrega 1 - Comercializar Eventos				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
1	Comercial	Crear un evento confirmado	Hacer el seguimiento del mismo	- Debe tener Nombre, Fecha, Descripción, Destinatarios, Programa, Instructor, Lugar, Ciudad, País, Capacidad, Precios y Promociones: SEB (Super Early Bird), EB (Early Bird), dto. en % para 2 personas y dto. en % para 3 o más personas. - Las promociones son opcionales. - Las fechas de SEB y EB deben ser anteriores a la fecha del evento - Por defecto SEB=30 días antes, EB=10 días antes, 2personas=-10%, 3+ personas=-15%. - Un evento puede ser público o privado
2	Comercial	Ver listado de eventos confirmados	No superponer eventos	- Mostrar Nombre, Ciudad y País - Muestra solo los futuros - Ordenado por fecha ascendente
3	Comercial	Modificar evento confirmado	Corregir cualquier error o re programarlo	- Permite modificar todos los campos.
4	Comercial	Cancelar evento confirmado	Dejar de seguirlo	- Desaparece del listado de eventos confirmados.
5	Comercial	Listar los eventos en un sitio web	Que los interesados	- Solo se listan los eventos públicos - Listado por fechas (a futuro)

			puedan verlos	- Agrupados por Ciudad
6	Comercial	Publicar los detalles de cada evento	Que los interesados puedan verlos	- Accesible desde el listado de eventos - Muestra los detalles de cada evento: Nombre, Fecha, Descripción, Destinatarios, Programa, Instructor, Lugar, Ciudad, País, Precios y Promociones
7	Comercial	Generar un texto con fechas y valores	Pegarlo en los e-mail de respuesta	- Debe generarlo agrupando por ciudad, cursos, fechas y precios de cada uno.
8	Comercial	Dashboard de inscripciones a cursos	Conocer el estado de completitud de cada curso	- Muestra los eventos con colores: Rojo, Naranja, Amarillo y Verde. Los criterios son: → Un evento debe estar al 50% al menos 15 días antes → Un evento debe estar al 75% al menos una semana antes → Un evento debe estar al 100% dos días antes - La varianza sobre esos números alteran los colores: → Menos del 50% (Rojo) → Del 50% al 75% (Naranja) → Del 75% al 90% (Amarillo) → Del 90% al 100% (Verde)
9	Interesado	Pre-Inscribirme	Iniciar la reserva de mi vacante	Debe solicitar Nombre*, Apellido*, Teléfono de Contacto*, Email*, Empresa/Carrera, Rol y Solicitar confirmación de que el asistente llevará notebook* si el curso lo requiere. * = obligatorio, el resto, opcional.
10	Comercial	Ser notificado de cada inscripción	Poder reaccionar en tiempo real frente a cada una	El email debe ser enviado a una dirección de correo configurable indicando los datos de contacto de la persona que realizó la inscripción.
11	Comercial	Confirmar la inscripción sin pago (pago a cuenta)	Financiar ciertas vacantes	Un pre-inscripto puede convertirse en inscripto sin haber realizado el pago.
12	Comercial	Conocer los	Realizar el	Listar los eventos con pagos

		Pagos Pendientes por evento	seguimiento de los pagos	pendientes y un detalle de las pre-inscripciones pendientes de pago por cada evento.
13	Interesado	Pagar en efectivo	Confirmar mi vacante	Una vez que un interesado se pre-inscribe debe proporcionarle los datos para poder pagar en efectivo.
14	Interesado	Pagar con Cheque	Confirmar mi vacante	Una vez que un interesado se pre-inscribe debe proporcionarle los datos para poder pagar con cheque.
15	Interesado	Pagar por Transferencia Bancaria	Confirmar mi vacante	Una vez que un interesado se pre-inscribe debe proporcionarle los datos para poder pagar por transferencia bancaria.
16	Comercial	Registrar los Pagos	Realizar el seguimiento de los pagos	Una pre-inscripción puede convertirse en inscripción registrando el pago realizado (fecha, monto y forma de pago).
17	Gestor de Cobranzas	Ser notificado del cobro de un evento	Realizar el seguimiento de los pagos	Cada vez que una pre-inscripción se convierte en inscripción, se debe enviar un e-mail a una casilla configurable.

Entrega 2 - Toma de evaluaciones on-line				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
18	Alumno	Responder Preguntas Multiple-Choice	Rendir el examen final	- Si el evento requiere un examen final, el alumno debería poder responder las preguntas on-line (solo multiple-choice) - Se deben poder administrar exámenes, preguntas y sus posibles respuestas. - Se asigna un tipo de examen a los alumnos seleccionados de un determinado evento.
19	Alumno	Un aviso de Finalización de Evaluación	Para saber que he finalizado	Se avisará en pantalla
20	Instructor	Que se realice la corrección automática de preguntas multiple-choice	Reducir mi carga de trabajo post-evento	- Todo examen debe tener un puntaje mínimo requerido (expresado en porcentaje) - Siendo preguntas multiple-choice, las correctas suman un punto, las incorrectas no suman ni restan.
21	Alumno	Recibir una notificación del resultado por e-mail	Para conocer el resultado de mi examen	- Se notifica por e-mail al alumno tan pronto finalice el examen.
22	Alumno	Generar mi certificado de evaluación aprobada	Presentarlo donde sea necesario	- El alumno podrá bajar un PDF son la constancia de su aprobación de examen
23	Instructor	Conocer los recuperatorios pendientes	Hacer seguimiento con los alumnos	- Para un determinado evento, se listarán los exámenes y recuperatorios pendientes.
24	Alumno	Conocer las preguntas erradas	Con el fin de saber dónde he fallado mi evaluación	- Se listarán las preguntas correctas con su explicación y las preguntas erradas, sin explicación.
25	Alumno	Recuperar las preguntas erradas	Con el fin de aprobar el examen	- Solo se realizará la respuesta de las preguntas erradas - Al finalizar el recuperatorio, aplican las mismas acciones que para un

				examen estándar: aviso de finalización, corrección automática y aviso de resultado

Entrega 3 - Pre-Inscripción Individual y Corrección de Exámenes a Desarrollar				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
26	Interesado	Recibir un aviso de Pre-Inscripción y pasos siguientes	Poder confirmar mi pre-inscripción	- Al inscribirse, el alumno recibe por e-mail un instructivo sobre los pasos a seguir para efectivizar su inscripción.
27	Alumno	Responder Preguntas a Desarrollar	Poder rendir el examen	- Ciertas preguntas deberán solicitar un texto libre como respuesta.
28	Instructor	Listado de evaluaciones a corregir	Poder corregir evaluaciones	- Se deberán listar las evaluaciones con preguntas a desarrollar que estén pendientes de corrección
29	Instructor	Corrección manual de preguntas a desarrollar	Poder calificar a los alumnos	- Las respuestas de texto libre deberán ser corregidas manualmente por el instructor
30	Instructor	Feedback de corrección	Poder recomendar o sugerir acciones a los alumnos	- Al finalizar la corrección, el instructor podrá dar feedback de la evaluación por medio de un campo de texto libre.

Entrega 4 - Pre-Inscripción Corporativa y Seguimiento de Pagos				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
31	Empresa	Realizar una Pre-Inscripción corporativa	Inscribir varios empleados de una sola vez	Al inscribir se deberá solicitar responsable (mismos campos que pre-inscripción) y cantidad de inscriptos (hasta 10). - Luego, cada inscripto deberá tener Nombre, Apellido, E-mail y Número de Contacto.
32	Interesado	Recibir un Recordatorio de Evento	Alertarme sobre la proximidad del evento e informarme sobre los pormenores	- Enviado por e-mail al e-mail de contacto de cada inscripto (copiando a los responsables de inscripciones corporativas una sola vez). - Recordar horario, lugar y requisitos. Solicitar aviso en el caso de que el evento tenga almuerzo y el interesado tenga restricciones alimenticias. - Se debe enviar dos días antes del evento.
33	Responsable Logístico	Ser notificado sobre cupo alcanzado	A definir	A definir
34	Interesado	Ser notificado sobre el pago pendiente	Poder confirmar mi vacante a tiempo	- Se enviará un recordatorio de pago pendiente 48hs luego de la pre-inscripción, avisando que la misma vence en 24hs hábiles.
35	Administrativo	Obtener la información de facturación	Poder emitir las facturas correctamente	- Con cada Pre-Inscripción se solicitará información de facturación: Razón Social, Domicilio Fiscal, CUIT, Situación frente al IVA.
36	Interesado	Pagar por PayPal	Confirmar mi Vacante	- El sistema deberá proveer un link de pago de PayPal.
37	Interesado	Pagar por MercadoPago	Confirmar mi Vacante	- El sistema deberá proveer un link de pago de MercadoPago.

Entrega 5 - Logística de Eventos				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
38	Responsable de Logística	Gestionar diferentes Tipos de Eventos	Crear checklists de cada tipo	- ABM de Tipos de Evento. Solo Nombre y Descripción.
39	Responsable de Logística	Gestionar diferentes modelos de Checklist	Que cada evento pueda instancias su checklist en base a un modelo pre armado	- Uno por cada tipo de evento
40	Responsable de Logística	Gestionar diferentes listados de Materiales	Saber qué se debe comprar por cada evento	- Un listado de materiales por cada tipo de evento.
41	Responsable de Logística	Hacer el seguimiento de cada Checklist de Evento	Que el mismo se realice de forma eficiente	- Poder marcar como “cumplido” los hitos de un checklist y dejar anotaciones (opcionales).
42	Responsable de Logística	Modificar los datos de un Checklist	Tener flexibilidad a la hora de gestionar un evento	- se podrán agregar o eliminar hitos de un checklist de evento particular.
43	Responsable de Logística	Ser notificado al modificar un checklist	Para estar al tanto de las modificaciones	- Se enviará un e-mail al responsable de logística con cada modificación de checklist (no incluye al avance del checklist de evento).
44	Responsable de Logística	Conocer los eventos y el progreso de checklist de cada uno	Para asegurar el correcto seguimiento de los checklists	- Listar los eventos y el porcentaje de avance de cada checklist
45	Responsable de Logística	Detalle de checklist de evento	Para asegurar el correcto seguimiento de los checklists	- Detallar el estado de cada checklist con hitos cumplidos y pendientes y fechas esperadas por cada uno.

Entrega 6 - Eventos Tentativos				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
46	Partner Comercial	Crear evento tentativo	Proponer la realización del mismo	A definir
47	Comercial	Ser notificado de un nuevo evento tentativo	Realizar las acciones necesarias para la confirmación del mismo	A definir
48	Partner Comercial	Consultar agenda de eventos	Conocer las fechas y disponibilidad para crear eventos tentativos	A definir
49	Partner Comercial	Modificar evento tentativo	Realizar correcciones o reprogramar eventos tentativos	A definir
50	Partner Comercial	Cancelar evento tentativo	Dejar de seguirlo	A definir
51	Comercial	Ver listado de eventos tentativos	Tener un panorama de la planificación futura de eventos	A definir
52	Comercial	Confirmar evento tentativo	Transformarlo en un evento agendado y publicarlo.	A definir
53	Partner Comercial / Comercial / Instructor	Ser notificado sobre la confirmación de evento	Comenzar a comercializarlo	A definir
54	Comercial	Ver listado de eventos tentativos agrupados por Partner Comercial y/o Región	Tener un panorama de la planificación futura de eventos	A definir
55	Interesado	Obtener un brochure de cada evento	Evaluar la información con mayor detalle	A definir
56	Interesado	Pre-Inscribir un grupo de personas	Asistir varios a un mismo evento sin ser una organización	A definir

Entrega 7 – Integración con sistemas Externos				
Prio.	Como ...	Necesito ...	Para ...	Criterios de Aceptación
57	Partner Comercial	Consultar agenda de instructores	Conocer su disponibilidad	A definir
58	Comercial	Registrar evento tentativo en Google Calendar	Publicar su existencia a todos los suscriptos a dicho calendario	A definir
59	Comercial	Registrar evento confirmado en Google Calendar	Publicar su existencia a todos los suscriptos a dicho calendario	A definir
60	Responsable Financiero	Crear balance contable del evento en Google Docs	Comenzar a hacer el seguimiento financiero de un evento	A definir
61	Comercial	Publicar Evento en Twitter, Facebook & LinkedIn	Dar a conocer su existencia	A definir
62	Comercial	Difundir vía Mailchimp	Dar a conocer su existencia	A definir
63	Comercial	Difundir en forma masiva	Dar a conocer su existencia	A definir
64	Comercial	Difundir a leads comerciales	Dar a conocer su existencia	A definir
65	Administrativo	La generación de asiento contable de Cobro	Registrar el cobro con menor esfuerzo	A definir
66	Administrativo	Generar e Imprimir Factura	Reducir mi esfuerzo y probabilidad de error	A definir
67	Administrativo	Ver listado de Facturas	Conocer las facturas generadas	A definir
68	Administrativo	Entregar Factura	Realizar el cobro de un evento	A definir
69	Administrativo	Asentar Factura en Contabilidad	Reducir mi esfuerzo y probabilidad de error	A definir

5. Estimaciones Ágiles

Cono de la Incertidumbre

En gestión de proyectos, el cono de la incertidumbre describe la evolución de la incertidumbre durante la ejecución de un proyecto. Al comienzo, poco es conocido sobre el producto y el resultado del trabajo, por tanto las estimaciones están sujetas a una gran incertidumbre. A medida que avanzamos en el proyecto obtenemos mayor conocimiento sobre el entorno, la necesidad de negocio, el producto y el proyecto mismo. Esto causa que la incertidumbre tienda a reducirse progresivamente hasta desaparecer, esto ocurre generalmente hacia el final del proyecto: no se alcanza una incertidumbre del 0% sino hasta haber finalizado.

Muchos ambientes cambian tan lentamente que la incertidumbre reinante puede ser considerada constante (evolución estática) durante la duración de un proyecto típico. En estos contextos la gestión tradicional de proyectos hace hincapié en lograr un entendimiento total mediante el análisis y la planificación detallada antes de comenzar a trabajar. De esta manera los riesgos son reducidos a un nivel en el que pueden ser gestionados cómodamente. En estas situaciones, el nivel de incertidumbre decrece rápidamente al comienzo y se mantiene prácticamente constante (y bajo) durante la ejecución del proyecto.

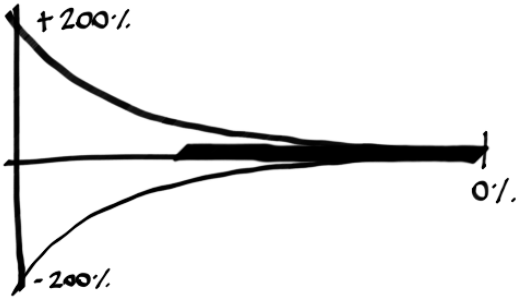


Ilustración 31: Cono de la Incertidumbre en un contexto estable

El contexto del software, por el contrario, es un contexto altamente volátil donde hay muchas fuerzas externas actuando para incrementar el nivel de incertidumbre, como lo son los cambios producidos en el contexto de negocio, los cambios tecnológicos y aquellos surgidos por la mera existencia del producto construido que acontecen durante la ejecución del proyecto. Debido a esta razón, se requiere trabajar activa y continuamente en reducir el nivel de incertidumbre.

Investigaciones han demostrado que en la industria del software, el nivel de incertidumbre al comienzo de un proyecto es del +/- 400%[27], esta incertidumbre tiende a decrementarse durante la evolución del proyecto, pero sin garantías de ello.

27 McConnell, S (2006) *Software Estimation: Demystifying the Black Art*, Microsoft Press.

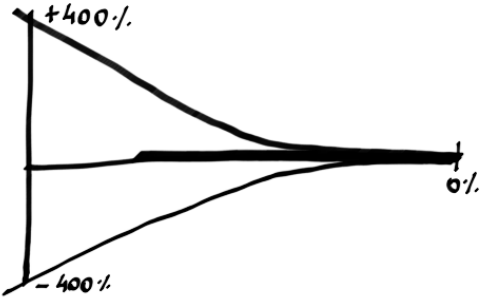


Ilustración 32: Cono de la Incertidumbre en un contexto inestable

Estimaciones en contextos inciertos

Como es de esperar según los gráficos previos, proveer una estimación precisa en etapas tempranas de un proyecto tiene como consecuencia un compromiso poco probable de ser cumplido.

A medida que adquiramos conocimiento, nuestras estimaciones se harán cada vez más precisas. El problema aparece a la hora de estimar cuando muchas de las decisiones se toman en base a supuestos que probablemente no sucedan o no sean los correctos.

"*La propia palabra "estimación" deja en claro que no calculamos un valor exacto, determinístico. De hecho, toda estimación tiene supuestos, y estos supuestos suman incertidumbres.*"[28]

28 Fontela, Carlos (2007) *Estimaciones y Estadística*, Blog CyS Ingeniería de Software

Como consecuencia, las metodologías ágiles proponen comenzar a trabajar en un proyecto sin la necesidad de tener una estimación precisa basada en supuestos y siendo conscientes de que la estimación inicial es de un orden de magnitud probable, para poder ganar experiencia rápidamente y así estimar con mayor certeza prescindiendo de supuestos.

Para mitigar el riesgo de proveer estimaciones incorrectas, en metodologías ágiles se opta por reducir la precisión de las estimaciones en función de cuánto conocimiento se tiene sobre el esfuerzo que se requiere estimar. De esta manera, los "requerimientos" y sus "estimaciones" se categorizan en diferentes niveles de precisión.

Escalas de PBIs y Estimaciones

Podemos enumerar la siguiente escala de PBIs y estimaciones:

- **Alto Nivel**: EPIC (bloque funcional) estimada en Tamaño (XS, S, M, L, XL)
- **Nivel Medio**: Historia de Usuario (funcionalidad) estimada en Puntos de Historia (Sucesión de Fibonacci[29])
- **Bajo Nivel**: tareas o actividades estimadas en horas, preferiblemente menos de un día.

Al comenzar el proyecto, nuestro *Product Backlog* se compone de bloques funcionales que podemos estimar según sus tamaños:

- XS – Muy Pequeño
- S – Pequeño

29 http://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci

- M – Medio
- L – Grande
- XL – Muy Grande

Esto nos permitirá tener una primera aproximación a la problemática de negocio y a las características del producto que se desea construir. Conociendo las prioridades de dichos bloques funcionales, se toman los de mayor prioridad y se descomponen en funcionalidades más específicas, logrando de esa manera PBIs de menor nivel, llamados **Historias de Usuario** o *User Stories*. A las Historias de Usuario las estimaremos utilizando la sucesión Fibonacci:

- 0, 1, 2, 3, 5, 8, 13, 21, 40, 100[30]

Para estimar las Historias de Usuario utilizamos una técnica comparativa llamada **Estimación Relativa**. Esto significa asignar uno de los números de la serie de Fibonacci a cada una de las Historias de Usuario. De esta manera, aquellas historias que tengan el número 2 requerirán aproximadamente el doble de esfuerzo que las que lleven el número 1, aquellas que lleven el número 3 requerirán aproximadamente el triple de esfuerzo de las que lleven el número 1, una vez y media el esfuerzo de las que lleven el número 2, etc.

Finalmente llegamos al nivel más bajo de estimación: la estimación en horas. Solo aplica a las tareas o actividades de las Historias de Usuario que han sido seleccionadas para formar parte de un determinado Sprint. En la reunión de planificación

30 Con una leve deformación ya que se interrumpe la sucesión en el número 21 y se agregan luego los números 40 y 100.

de dicho Sprint, estas Historias de Usuario son divididas por el Equipo en tareas o actividades y a su vez, las tareas o actividades, estimadas en horas. Lo importante es que la estimación en horas solo se realiza para un las actividades de un determinado Sprint.

Autores como Jeff Sutherland[31] expresan no estar de acuerdo con estimar a este bajo nivel, mientras otros como Mike Cohn[32] promueven su utilización. Esta situación da origen a dos modelos de planificación de Sprint: la planificación basada en velocidad (*velocity-based planning*) donde el Equipo se compromete a realizar tantos User Stories de modo que sumen una estimación en Puntos de Historia igual a la velocidad del Equipo, por un lado, y la planificación basada en compromisos (*commitment-based planning*) donde sin importar la velocidad, el Equipo revalúa las Historias de Usuario y se compromete en función de la estimación de cada una de ellas, por el otro. En este último caso, las Historias de Usuario involucradas deberían sumar una cantidad de Puntos de Historia aproximada a la Velocidad del Equipo. Dice Mike Cohn con respecto a la utilización de la Velocidad del Equipo como herramienta de planificación:

"*Supongamos que un equipo de básquetbol está en la mitad de su temporada. Han anotado un promedio de 98 puntos por cada partido de los 41 partidos jugados hasta el momento. Sería conveniente para ellos decir "Nosotros anotaremos un promedio*

31 Jeff Sutherland (2010),"*Story Points: Why are they better than hours?*", http://scrum.jeffsutherland.com/2010/04/story-points-why-are-they-better-than.html

32 Mike Cohn (2007), "*Why I Don't use Story Points for Sprint Planning*", http://blog.mountaingoatsoftware.com

de 98 puntos por partido por el resto de la temporada." Pero no deberían nunca decir antes de cualquier partido "Nuestro promedio es de 98 puntos, por lo que se anotarán 98 puntos esta noche." Es por eso que afirmo que la velocidad es un predictor útil para el largo plazo, pero no lo es para el corto plazo."[33]

Métodos Delphi de Predicción y Estimación

El **Método Delphi** es una técnica creada por la Corporación RAND[34] hacia fines de la década de los 40's para la elaboración de pronósticos y predicciones sobre el impacto de la tecnología en la Guerra Fría.

Su objetivo es lograr un consenso basado en la discusión entre expertos. Este método se basa en la elaboración de un cuestionario que ha de ser contestado por una serie de expertos. Una vez recibida la información, se vuelve a realizar otro cuestionario basado en el anterior para ser contestado nuevamente.

Al final, el responsable del estudio elaborará sus conclusiones a partir de la explotación estadística de los datos obtenidos en las iteraciones anteriores.

El Método Delphi se basa en:

- El anonimato de los participantes

33 *Ibid.*

34 La Corporación RAND (Research And Development) es un laboratorio de ideas (think tank) norteamericano formado, en un primer momento, para ofrecer investigación y análisis a las fuerzas armadas norteamericanas. (fuente: Wikipedia)

- La repetición y retroalimentación controlada
- La respuesta del grupo en forma estadística

Basados en el Método Delphi, Barry Boehm y John Farquhar elaboraron en 1970 la variante conocida desde entonces como **Wideband Delphi**. Se trata de una técnica basada en la obtención de consensos para la estimación de esfuerzos, llamada "wideband" porque a diferencia del conocido método Delphi, esta técnica requiere de un mayor grado de interacción y discusión entre los participantes. Wideband Delphi fue popularizado en 1981 por Boehm en su libro "*Software Engineering Economics*" donde presenta los siguientes pasos para su ejecución:

- Un coordinador presenta a cada experto una especificación y un formulario de estimación.
- El coordinador convoca a una reunión de grupo en la que los expertos debaten temas de estimación.
- Los expertos llenan los formularios de forma anónima.
- El coordinador prepara y distribuye un resumen de las estimaciones.
- El coordinador convoca a una reunión de grupo, centrándose específicamente en aquellas estimaciones donde los expertos varían ampliamente.
- Los expertos completan los formularios una vez más de forma anónima, y los pasos 4 a 6 son repetidos para tantas rondas como sea necesario.

Planning Poker

James Greening presentó en su paper en 2002 llamado "*Planning Poker (o cómo evitar análisis parálisis en la*

planificación de liberaciones)"[35] donde se basa en el método Wideband Delphi para realizar la estimación de requerimientos (o User Stories) de forma colaborativa en un Equipo. La técnica consiste en que cada integrante del Equipo posee en sus manos una baraja de cartas con los números correspondientes a la sucesión de Fibonacci[36] y se siguen los siguientes pasos:

- El responsable del negocio presenta una historia de usuario para ser estimada.
- Todos los participantes proceden a realizar su estimación en forma secreta, sin influenciar al resto del Equipo, poniendo su carta elegida boca abajo sobre la mesa.
- Una vez que todos los integrantes han estimado, se dan vuelta las cartas y se discuten principalmente los extremos.
- Al finalizar la discusión se levantan las cartas y se vuelve a estimar, esta vez con mayor información que la que se tenía previamente.
- Las rondas siguen hasta que se logra consenso en el Equipo y luego se continúa desde el punto número uno con una nueva historia de usuario.

Este método fue popularizado en 2005 por Mike Cohn en su libro "*Agile Estimating and Planning*".

35 http://renaissancesoftware.net/files/articles/PlanningPoker-v1.1.pdf
36 Se puede repasar en la sección de Escalas de PBIs y Estimaciones

La Sabiduría de las Multitudes (Wisdom of Crowds)

James Surowieki explica en su libro "La Sabiduría de las Multitudes" (2004): "*Normalmente solemos favorecer la opinión de los expertos, pues consideramos que sólo una persona con experiencia y conocimientos suficientes es capaz de emitir juicios correctos en un área o materia en particular. Sin embargo, hay evidencias de que las decisiones tomadas colectivamente por un grupo de personas suelen ser más atinadas que las decisiones tomadas sobre la base del conocimiento de un experto*".

La tesis detrás de la Sabiduría de las Multitudes es simple: dadas las circunstancias requeridas, un grupo de personas puede tomar una decisión más acertada que la mejor de las decisiones de la mayoría (si no todos) los integrantes del grupo individualmente.

Para que esto pueda suceder, Surowieki recomienda en su tesis las siguientes condiciones:

- **Diversidad de opiniones**: cada persona debería tener información particular aún si es sólo una interpretación excéntrica de los hechos conocidos. El grupo debe tener diversidad de perfiles.
- **Independencia**: las opiniones de los participantes no deberían estar influenciadas por las opiniones de los que los rodean, con el objetivo de evitar el Pensamiento de Grupo[37].

37 http://es.wikipedia.org/wiki/Pensamiento_de_grupo

- **Agregación**: El grupo debería tener la capacidad de sumar las opiniones individuales y no simplemente votar por la mejor opción.

Conclusiones sobre estimaciones Ágiles

Muchas teorías y enfoques convergen en las siguientes características sobre estimaciones en proyectos ágiles:

- No tiene sentido presentar estimaciones certeras al comienzo de un proyecto ya que su probabilidad de ocurrencia es extremadamente baja por el alto nivel de incertidumbre.
- Intentar bajar dicha incertidumbre mediante el análisis puede llevarnos al "Análisis Parálisis"[38]. Para evitar esto debemos estimar a alto nivel con un elevado grado de probabilidad, actuar rápidamente, aprender de nuestras acciones y refinar las estimaciones frecuentemente. Este enfoque se conoce también como "Rolling Wave Planning" o "Elaboración Progresiva".
- La mejor estimación es la que provee el Equipo de trabajo. Esta estimación será mucho más realista que la estimación provista por un experto ajeno al Equipo.

38 http://en.wikipedia.org/wiki/Analysis_paralysis

6. Plan de Entregas (*Release Plan*)

A continuación se presentan las Historias de Usuario estimadas por el Equipo de Desarrollo, utilizando Planning Poker con Fibonacci y estimando una velocidad de Iteración de 15 puntos de historia con una duración de dos semanas:

Entrega 1 - Comercializar Eventos				
Prio.	Como ...	Necesito ...	Para ...	Estim.
Sprint 1 – Velocidad: 15 puntos				
1	Comercial	Crear un evento confirmado	Hacer el seguimiento del mismo	3
2	Comercial	Ver listado de eventos confirmados	No superponer eventos	2
3	Comercial	Modificar evento confirmado	Corregir cualquier error o re programarlo	2
4	Comercial	Cancelar evento confirmado	Dejar de seguirlo	1
5	Comercial	Listar los eventos en un sitio web	Que los interesados puedan verlos	2
6	Comercial	Publicar los detalles de cada evento	Que los interesados puedan verlos	5
Sprint 2 – Velocidad: 15 puntos				
7	Comercial	Generar un texto con fechas y valores	Pegarlo en los e-mail de respuesta	5
9	Interesado	Pre-Inscribirme	Iniciar la reserva de mi vacante	2
8	Comercial	Dashboard de inscripciones a cursos	Conocer el estado de completitud de cada curso	8
Sprint 3 – Velocidad: 14 puntos				

10	Comercial	Ser notificado de cada inscripción	Poder reaccionar en tiempo real frente a cada una	2
11	Comercial	Confirmar la inscripción sin pago (pago a cuenta)	Financiar ciertas vacantes	2
12	Comercial	Conocer los Pagos Pendientes por evento	Realizar el seguimiento de los pagos	3
13	Interesado	Pagar en efectivo	Confirmar mi vacante	1
14	Interesado	Pagar con Cheque	Confirmar mi vacante	1
15	Interesado	Pagar por Transferencia Bancaria	Confirmar mi vacante	1
16	Comercial	Registrar los Pagos	Realizar el seguimiento de los pagos	2
17	Gestor de Cobranzas	Ser notificado del cobro de un evento	Realizar el seguimiento de los pagos	2
Entrega 2 - Toma de evaluaciones on-line				
Prio.	Como ...	Necesito ...	Para ...	Estim.
Sprint 4 – Velocidad: 15 puntos				
18	Alumno	Responder Preguntas Multiple-Choice	Rendir el examen final	8
19	Alumno	Un aviso de Finalización de Evaluación	Para saber que he finalizado	2
20	Instructor	Que se realice la corrección automática de preguntas multiple-choice	Reducir mi carga de trabajo post-evento	5
Sprint 5 – Velocidad: 15 puntos				
21	Alumno	Recibir una notificación del resultado por e-mail	Para conocer el resultado de mi examen	2
22	Alumno	Generar mi certificado de evaluación aprobada	Presentarlo donde sea necesario	5
23	Instructor	Conocer los	Hacer seguimiento con	3

		recuperatorios pendientes	los alumnos	
25	Alumno	Recuperar las preguntas erradas	Con el fin de aprobar el examen	5
Sprint 6 – Velocidad: 15 puntos				
24	Alumno	Conocer las preguntas erradas	Con el fin de saber dónde he fallado mi evaluación	5
Entrega 3 - Pre-Inscripción Individual y Corrección de Exámenes a Desarrollar				
Prio.	Como ...	Necesito ...	Para ...	Estim.
26	Interesado	Recibir un aviso de Pre-Inscripción y pasos siguientes	Poder confirmar mi pre-inscripción	2
27	Alumno	Responder Preguntas a Desarrollar	Poder rendir el examen	8
Sprint 7 – Velocidad: 16 puntos				
28	Instructor	Listado de evaluaciones a corregir	Poder corregir evaluaciones	3
29	Instructor	Corrección manual de preguntas a desarrollar	Poder calificar a los alumnos	3
30	Instructor	Feedback de corrección	Poder recomendar o sugerir acciones a los alumnos	2
Entrega 4 - Pre-Inscripción Corporativa y Seguimiento de Pagos				
Prio.	Como ...	Necesito ...	Para ...	Estim.
31	Empresa	Realizar una Pre-Inscripción corporativa	Inscribir varios empleados de una sola vez	8
Sprint 8 – Velocidad: 15 puntos				
32	Interesado	Recibir un Recordatorio de Evento	Alertarme sobre la proximidad del evento e informarme sobre los pormenores	3
33	Responsabl	Ser notificado sobre cupo	A definir	2

	e Logístico	alcanzado		
34	Interesado	Ser notificado sobre el pago pendiente	Poder confirmar mi vacante a tiempo	3
35	Administrativo	Obtener la información de facturación	Poder emitir las facturas correctamente	2
36	Interesado	Pagar por PayPal	Confirmar mi Vacante	2
37	Interesado	Pagar por MercadoPago	Confirmar mi Vacante	3
Entrega 5 - Logística de Eventos				
Prio.	Como ...	Necesito ...	Para ...	Estim.
Sprint 9 – Velocidad: 15 puntos				
38	Responsable de Logística	Gestionar diferentes Tipos de Eventos	Crear checklists de cada tipo	2
39	Responsable de Logística	Gestionar diferentes modelos de Checklist	Que cada evento pueda instancias su checklist en base a un modelo pre armado	8
40	Responsable de Logística	Gestionar diferentes listados de Materiales	Saber qué se debe comprar por cada evento	5
Sprint 10 – Velocidad: 15 puntos				
41	Responsable de Logística	Hacer el seguimiento de cada Checklist de Evento	Que el mismo se realice de forma eficiente	5
42	Responsable de Logística	Modificar los datos de un Checklist	Tener flexibilidad a la hora de gestionar un evento	8
43	Responsable de Logística	Ser notificado al modificar un checklist	Para estar al tanto de las modificaciones	2
Sprint 11 – Velocidad: 15 puntos				
44	Responsabl	Conocer los eventos y el	Para asegurar el correcto	3

	e de Logística	progreso de checklist de cada uno	seguimiento de los checklists	
45	Responsable de Logística	Detalle de checklist de evento	Para asegurar el correcto seguimiento de los checklists	3
Entrega 6 - Eventos Tentativos				
Prioridad	Como ...	Necesito ...	Para ...	Estimación
46	Partner Comercial	Crear evento tentativo	Proponer la realización del mismo	3
47	Comercial	Ser notificado de un nuevo evento tentativo	Realizar las acciones necesarias para la confirmación del mismo	2
48	Partner Comercial	Consultar agenda de eventos	Conocer las fechas y disponibilidad para crear eventos tentativos	3
49	Partner Comercial	Modificar evento tentativo	Realizar correcciones o reprogramar eventos tentativos	1
Sprint 12 – Velocidad: 16 puntos				
50	Partner Comercial	Cancelar evento tentativo	Dejar de seguirlo	2
51	Comercial	Ver listado de eventos tentativos	Tener un panorama de la planificación futura de eventos	2
52	Comercial	Confirmar evento tentativo	Transformarlo en un evento agendado y publicarlo.	2
53	Partner Comercial / Comercial / Instructor	Ser notificado sobre la confirmación de evento	Comenzar a comercializarlo	2
54	Comercial	Ver listado de eventos tentativos agrupados por	Tener un panorama de la planificación futura de	3

		Partner Comercial y/o Región	eventos	
55	Interesado	Obtener un brochure de cada evento	Evaluar la información con mayor detalle	5
Sprint 13 – Velocidad: 16 puntos				
56	Interesado	Pre-Inscribir un grupo de personas	Asistir varios a un mismo evento sin ser una organización	8
Entrega 7 – Integración con sistemas Externos				
Prio.	Como ...	Necesito ...	Para ...	Estim.
57	Partner Comercial	Consultar agenda de instructores	Conocer su disponibilidad	3
58	Comercial	Registrar evento tentativo en Google Calendar	Publicar su existencia a todos los suscriptos a dicho calendario	5
Sprint 14 – Velocidad: 15 puntos				
59	Comercial	Registrar evento confirmado en Google Calendar	Publicar su existencia a todos los suscriptos a dicho calendario	5
60	Responsable Financiero	Crear balance contable del evento en Google Docs	Comenzar a hacer el seguimiento financiero de un evento	5
61	Comercial	Publicar Evento en Twitter, Facebook & LinkedIn	Dar a conocer su existencia	5
Sprint 15 – Velocidad: 15 puntos				
62	Comercial	Difundir vía Mailchimp	Dar a conocer su existencia	5
63	Comercial	Difundir en forma masiva	Dar a conocer su existencia	5
64	Comercial	Difundir a leads comerciales	Dar a conocer su existencia	5

Sprint 16 – Velocidad: 15 puntos				
65	Administrativo	La generación de asiento contable de Cobro	Registrar el cobro con menor esfuerzo	8
66	Administrativo	Generar e Imprimir Factura	Reducir mi esfuerzo y probabilidad de error	5
67	Administrativo	Ver listado de Facturas	Conocer las facturas generadas	3
Sprint 17 – Velocidad: 11 puntos				
68	Administrativo	Entregar Factura	Realizar el cobro de un evento	3
69	Administrativo	Asentar Factura en Contabilidad	Reducir mi esfuerzo y probabilidad de error	8

Incepción

La incepción es una aproximación que muchos autores utilizan para realizar todas aquellas tareas necesarias para hacer el setup de un proyecto de desarrollo. Esto incluye pero no se limita únicamente a configurar los entornos de desarrollo, realizar el plan de entregas, diseñar la arquitectura de la aplicación a alto nivel, configurar el repositorio de código fuente, socializar una visión común entre equipo Scrum y stakeholders, crear un elevator pitch del producto, realizar un impact mapping y un visual story mapping, etc. En nuestro caso, la Incepción tendrá una duración de 2 semanas, aunque podría ser diferente a los Sprints de desarrollo.

Duración del Proyecto

Duración Total: 18 Sprints = 36 Semanas = 9 meses

Etapa	Duración	Desde	Hasta
Incepción	2 semanas	3-Oct-2011	14-Oct-2011
Entrega 1 - Comercializar Eventos			
Sprint 1	2 semanas	17-Oct-2011	28-Oct-2011
Sprint 2	2 semanas	31-Oct-2011	11-Nov-2011
Sprint 3	2 semanas	14-Nov-2011	25-Nov-2011
Entrega 2 - Toma de evaluaciones on-line			
Sprint 4	2 semanas	28-Nov-2011	9-Dic-2011
Sprint 5	2 semanas	12-Dic-2011	23-Dic-2011
Sprint 6	2 semanas	26-Dic-2011	6-Ene-2012
Entrega 3 - Pre-Inscripción Individual y Corrección de Exámenes a Desarrollar			
Sprint 7	2 semanas	9-Ene-2012	20-Ene-2012
Entrega 4 - Pre-Inscripción Corporativa y Seguimiento de Pagos			
Sprint 8	2 semanas	23-Ene-2012	3-Feb-2012
Entrega 5 - Logística de Eventos			
Sprint 9	2 semanas	6-Feb-2012	17-Feb-2012
Sprint 10	2 semanas	20-Feb-2012	2-Mar-2012
Sprint 11	2 semanas	5-Mar-2012	16-mar-2012
Entrega 6 - Eventos Tentativos			
Sprint 12	2 semanas	19-Mar-2012	30-mar-2012
Sprint 13	2 semanas	2-Abr-2012	13-Abr-2012
Entrega 7 – Integración con sistemas Externos			
Sprint 14	2 semanas	16-Abr-2012	27-Abr-2012
Sprint 15	2 semanas	30-Abr-2012	11-May-2012
Sprint 16	2 semanas	14-May-2012	25-May-2012
Sprint 17	2 semanas	28-May-2012	8-Jun-2012

7. Costo del Proyecto

Para la realización de este proyecto se ha conformado un equipo de trabajo con las siguientes características y costos:

Perfil	Precio por Hora
Product Owner	$250.-/hr.
ScrumMaster	$200.-/hr.
Desarrolladores (3)	$170 c/u = $510.-/hr.
Total Equipo	$960.-/hr.
Concepto	**Sub-Total Proyecto**
9 meses = 1440 horas del equipo	$1.382.400.-
5 notebooks 4GB RAM c/u	$20.000.-
Servidor de Testing/UAT ($450.-/mes)	$4.050.-
Servidor de Integración Continua ($450.-/mes)	$4.050.-
Servidor de Repositorio de Código Fuente ($450.-/mes)	$4.050.-
Alquiler de Oficina Mensual ($4000.-/mes)	$36.000.-
Conectividad (Internet) ($380.-/mes)	$3.420.-
Comunicaciones (Celular) ($580.-/mes)	$5.220.-
Fondo de Contingencia (10%)	$145.919.-
Total del Proyecto:	$1.605.109.-

Bibliografía Recomendada

ADZIC, G., 2012, *Impact Mapping: Making a big impact with software products and projects*, Provoking Thoughts

COHN, M., 2004, *User Stories Applied: For Agile Software Development*, Addison-Wesley Professional

COHN, M., 2005, *Agile Estimating and Planning*, Prentice Hall

COHN, M., 2009, *Succeeding with Agile: Software Development Using Scrum*, Addison-Wesley Professional

PATTON, J., 2008, *User Story Mapping, http://www.agileproductdesign.com*

PICHLER, R., 2010, *Agile Product Management with Scrum: Creating Products that Customers Love*, Addison-Wesley Professional

PINK, D., 2010, *Drive: The Surprising Truth About What Motivates Us*, Canongate Books

RASMUSSON, J., 2013, *The Agile Samurai: How Agile Masters Deliver Great Software*, Pragmatic Bookshelf

RIES, E., 2011, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*, Crown Business

SCHWABER, K.; BEEDLE, M., 2001, *Agile Software Development with Scrum*, Prentice Hall

Acerca del Autor

Me desempeño como coach organizacional y entrenador profesional. Como Certified Scrum Coach (CSC) y Certified Scrum Trainer (CST), mi principal área de intervención es el trabajo en equipo bajo un marco colaborativo y relacional conocido como Scrum dentro del ámbito de del desarrollo de productos tecnológicos. Mi principal inquietud hoy en día pasa por asistir a otros rubros, más allá del tecnológico, a percibir y capitalizar los beneficios de esta nueva propuesta de trabajo, haciendo énfasis en las relaciones interpersonales y las competencias de los miembros de las organizaciones que pretenden adoptarla.

Crecí como profesional en el rubro tecnológico, tuve la oportunidad de formar parte de empresas pequeñas, medianas y organizaciones multinacionales con más de 200.000 empleados.

En el 2009 fundé Kleer, empresa de la que hoy formo parte. Junto a profesionales a los que admiro, día a día, llevamos adelante nuestra organización con el objetivo de promover y asistir a otras organizaciones a adoptar nuevas y más efectivas formas de trabajar y técnicas de creación de productos y servicios innovadores.

"Mi compromiso es asistir a los equipos y las organizaciones a alcanzar esos resultados que aun no alcanzaron"

Martín Alaimo
http://www.martinalaimo.com
martin.alaimo@kleer.la

Acerca de Kleer

Somos una empresa de capacitación y coaching. Creemos en una forma de trabajo clara, centrada en las personas y orientada hacia las necesidades específicas de cada contexto.

Nuestras premisas son:

- **Mantenerlo simple**. Las metodologías y prácticas de trabajo en las que confiamos aportan claridad a los proyectos. Los proyectos claros se vuelven más previsibles y con menor incidencia de errores evitables.
- **Las personas son todo**. No acompañamos proyectos sino equipos y personas. Nuestro propósito es transmitirles nuestro conocimiento y experiencia para que logren resultados de los que se sientan orgullosos.
- **Cada contexto es un mundo**. Estudiamos las características de cada proyecto y organización para entender cuáles son las prácticas y metodologías que mejor responden a sus necesidades y objetivos de negocio.

"*Creemos que hay otras formas de relacionarnos para conseguir resultados fantásticos que nos hagan sentir orgullosos*"

Kleer.la
http://www.kleer.la
entrenamos@kleer.la